

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Jure Leskovec, Anand Rajaraman and Ullman, J.D. (2020). Mining of massive datasets. Cambridge Cambridge University Press. (open access at <http://www.mmds.org/>)

- **Required pages:** The entire content of "3.5 Distance Measures" from chapter 3: <http://infolab.stanford.edu/~ullman/mmds/ch3n.pdf>

- **Estimated reading time:** 20 minutes

1. Daniel Jurafsky and James H. Martin. (2024). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models. Online manuscript released August 20, 2024. <https://web.stanford.edu/~jurafsky/slp3>

- **Required pages:** 2nd edition: The entire "Chapter 6: Vector Semantics and Embeddings" https://web.stanford.edu/~jurafsky/slp3/old_jan25/6.pdf , or

3rd edition: The entire "Chapter 6: Embeddings" <https://web.stanford.edu/~jurafsky/slp3/5.pdf>

- **Estimated reading time:** 20 minutes

Lesson 1 - Similarity Measures

December 25, 2025

MODULE 4 | LESSON 1

1 Similarity Measures

Reading Time	45 minutes
Prior Knowledge	Basic statistics: Familiarity with concepts like mean, standard deviation, correlation, and distributions. Linear algebra: Basic knowledge of vectors, matrices, and operations like dot product. Probability: A basic understanding of probability concepts to aid in interpreting statistical measures and their significance. Financial and Investing Knowledge: Basic understanding of financial markets, instruments, and investment strategies; Familiarity with risk assessment and management concepts; Basic knowledge of sentiment analysis and its role in understanding market trends.
Keywords	Euclidean Distance, Manhattan Distance, Cosine Similarity, Jaccard Index, Word Mover's Distance (WMD), Dynamic Time Warping, Pearson Correlation, Spearman Rank Correlation, Term Frequency-Inverse Document Frequency (TF-IDF), Text Analysis, Natural Language Processing (NLP), Information Retrieval

In this lesson we explore similarity measures and their use in financial markets, especially Term Frequency-Inverse Document Frequency (TF-IDF) for analyzing text data like news and social media to gain insights for investment decisions. We will cover different types of similarity measures, benefits of TF-IDF, and its applications in sentiment analysis, risk management, and for other investment strategies.

```
[1]: # Load libraries
import numpy as np
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
```

1.1 1 Similarity Measures

Similarity measures are methods used to quantify how alike two data objects are. They are essential in various fields, including data mining, machine learning, and information retrieval. The choice of similarity measure depends on the type of data and the desired interpretation of similarity. Common types of Similarity Measures are:

Distance Measures: These measures quantify dissimilarity. Smaller distances indicate higher similarity:

- Euclidean distance
- Manhattan distance
- Minkowski distance
- Word Mover's Distance (WMD)
- Dynamic Time Warping

Similarity Coefficients: These measures directly quantify similarity. Higher values indicate greater similarity:

- Cosine similarity
- Jaccard index
- Pearson correlation coefficient
- Spearman Rank Correlation

Below we will consider each in further details.

1.1.1 1.1 Distance Measures

1. Euclidean distance

- **Type:** Distance measure
- **Data Type:** Numerical data
- **Description:** Measures the straight-line distance between two points in Euclidean space.
- **Formula:** $\sqrt{\sum (x_i - y_i)^2}$ (where x and y are the data points, and i represents the dimensions)
- **Interpretation:** Smaller distance = higher similarity
- **Use Cases:**
 - K-means clustering
 - K-nearest neighbors
 - Finding similar data points

2. Manhattan distance

- **Type:** Distance measure
- **Data Type:** Numerical data
- **Description:** Measures the distance between two points by summing the absolute differences of their coordinates. Also known as city block distance or L1 distance.

- **Formula:** $\sum |x_i - y_i|$
- **Interpretation:** Smaller distance = higher similarity
- **Use Cases:**
 - Recommender systems
 - Feature selection
 - Robust to outliers

3. Minkowski distance

- **Type:** Distance measure
- **Data Type:** Numerical data
- **Description:** A generalized distance metric that includes Euclidean and Manhattan distances as special cases.
- **Formula:** $(\sum |x_i - y_i|^p)^{1/p}$ (where p is the order of the distance)
 - $p = 1$: Manhattan distance
 - $p = 2$: Euclidean distance
- **Interpretation:** Smaller distance = higher similarity
- **Use Cases:**
 - Adaptable to different data distributions
 - Experimenting with different distance metrics

4. Word Mover's Distance (WMD)

- **Type:** Distance measure
- **Data Type:** Text data (documents, sentences)
- **Description:** Measures the dissimilarity between two text documents based on the “travel cost” of moving words from one document to match the words in the other document. Uses word embeddings to represent words in a semantic space.
- **Interpretation:** Lower WMD indicates higher similarity.
- **Use Cases:**
 - Document classification
 - Text similarity tasks where semantic meaning is important
 - Clustering documents

5. Dynamic Time Warping

- **Type:** Distance measure
- **Data Type:** Time series data
- **Description:** Measures the similarity between two time series that may vary in speed or have time shifts. It finds the optimal alignment between the time series by “warping” the time axis.
- **Interpretation:** Lower DTW distance indicates higher similarity.
- **Use Cases:**
 - Speech recognition
 - Gesture recognition
 - Financial time series analysis
 - Anomaly detection in time series

1.1.2 1.2 Similarity Coefficients

1. Cosine similarity

- **Type:** Similarity coefficient
- **Data Type:** Numerical vectors (often used for text data after vectorization)
- **Description:** Measures the cosine of the angle between two vectors.
- **Formula:** $(A \cdot B) / (\|A\| \|B\|)$ (where A and B are vectors, $\cdot$ is the dot product, and $\| \cdot \|$ represents magnitude)
- **Range:** -1 to 1 (1 = perfect similarity, 0 = no similarity, -1 = perfect dissimilarity)
- **Use Cases:**
 - Document similarity
 - Information retrieval
 - Recommender systems

2. Jaccard index

- **Type:** Similarity coefficient
- **Data Type:** Sets (categorical data)
- **Description:** Measures the similarity between two sets by dividing the number of elements they have in common by the total number of elements in both sets.
- **Formula:** $|A \cap B| / |A \cup B|$ (where $\cap$ is intersection and $\cup$ is union)
- **Range:** 0 to 1 (1 = perfect similarity, 0 = no similarity)
- **Use Cases:**
 - Text analysis (comparing documents based on word sets)
 - Image recognition (comparing image features)
 - Recommender systems

3. Pearson correlation coefficient

- **Type:** Similarity coefficient (but can be interpreted as a distance measure when considering 1 - correlation)
- **Data Type:** Numerical data
- **Description:** Measures the linear relationship between two variables.
- **Range:** -1 to 1 (1 = perfect positive correlation, 0 = no correlation, -1 = perfect negative correlation)
- **Use Cases:**
 - Finding relationships between variables
 - Feature selection
 - Stock market analysis

4. Spearman Rank Correlation

- **Type:** Similarity coefficient (but can be interpreted as a distance measure when considering 1 - correlation).
- **Data Type:** Ordinal data or numerical data where the rank order is more important than the actual values.
- **Description:** Measures the monotonic relationship between two variables. It assesses how well the relationship between two variables can be described by a monotonic function (always increasing or always decreasing).
- **Range:** -1 to 1, similar to Pearson correlation.
- **Use Cases:**
 - Assessing correlations when data doesn't meet the assumptions of Pearson correlation (linearity, normality).
 - Analyzing ranked data (e.g., customer preferences, survey responses).

1.1.3 1.3 Applications of similarity measures

Now let's expand on the context of similarity measures within the broader scope of data science and machine learning, as well as their relevance to financial applications.

Broader Context of Similarity Measures:

- **Data Mining and Machine Learning:** Similarity measures are fundamental building blocks in many data mining and machine learning algorithms. They are used for tasks like clustering (grouping similar data points), classification (assigning data points to categories based on similarity to existing categories), and anomaly detection (identifying unusual data points that differ significantly from others).
- **Information Retrieval:** In information retrieval systems, such as search engines, similarity measures are used to rank documents or web pages based on their relevance to a user's query. Documents that are more similar to the query are ranked higher in the search results.
- **Recommender Systems:** Recommender systems leverage similarity measures to suggest products, movies, or other items that are similar to those a user has liked or interacted with in the past.

Relevance to Financial Applications:

- **Financial Time Series Analysis:** Similarity measures like DTW are crucial for analyzing financial time series data, such as stock prices, exchange rates, or economic indicators. DTW can help identify patterns, trends, and correlations between different time series, even if they have variations in their temporal alignment. This information can be used for forecasting, risk management, and investment strategies.
- **Risk Management:** Similarity measures can be used to assess the similarity between different financial instruments or portfolios to identify potential risks and diversification opportunities. By measuring the similarity of risk profiles, investors can make more informed decisions about portfolio construction and risk mitigation.
- **Algorithmic Trading:** Similarity measures can be incorporated into algorithmic trading strategies to identify trading opportunities based on patterns and relationships between different financial assets. For example, a trading algorithm might identify stocks that have historically moved in similar ways and use this information to make trading decisions.

Overall, this lesson provides foundation for understanding the importance of similarity measures in various data science and machine learning applications, with a specific focus on their relevance to financial analysis. If we look on the broader context and potential applications of these measures, we can better appreciate their role in extracting insights and making informed decisions in the financial domain. Here are few examples of how similarity measures can be applied in financial engineering scenarios:

- **Clustering Financial Instruments:** Euclidean distance or correlation coefficients can be used to cluster stocks or other financial instruments based on their historical price movements or other financial metrics. This can help investors identify groups of assets that behave similarly and diversify their portfolios accordingly.
- **Detecting Anomalies in Financial Data:** Similarity measures can be used to detect anomalies or outliers in financial data, such as unusual trading activity or sudden price movements. These anomalies might indicate fraudulent activity or market disruptions.
- **Predicting Stock Price Movements:** Similarity measures can be used to build predictive models for stock price movements by identifying patterns and relationships between different

financial indicators or news sentiment.

1.1.4 1.4 Simplified examples

Let's now consider simplified example using financial data to illustrate the calculation of similarity using Euclidean distance, Manhattan distance, and cosine similarity. We will also delve into the geometry of these similarity measures.

Scenario: We have two stocks, Stock A and Stock B, and we want to measure their similarity based on their daily returns over a period of 5 days.

Day	Stock A Returns	Stock B Returns
1	0.02	0.03
2	-0.01	0.01
3	0.03	0.02
4	0.01	0.00
5	-0.02	-0.01

Let's represent this scenario data in code using Python lists or NumPy arrays, which will allow for programmatic calculations and easier manipulation in follow-up code snippets:

```
[2]: # Stock returns data
stock_a_returns = [0.02, -0.01, 0.03, 0.01, -0.02]
stock_b_returns = [0.03, 0.01, 0.02, 0.00, -0.01]

# Convert to NumPy arrays for easier calculations
stock_a = np.array(stock_a_returns)
stock_b = np.array(stock_b_returns)
```

Now, we have the daily returns of Stock A and Stock B stored in the variables `stock_a` and `stock_b`, respectively. These variables can be directly used in subsequent calculations.

Euclidean Distance: Euclidean distance represents the straight-line distance between two points in a multi-dimensional space. It's the shortest distance between the points, as if we were to draw a straight line connecting them. Imagine two points on a Cartesian plane. The Euclidean distance is the length of the line segment connecting these points. In higher dimensions, the concept extends similarly, but the line segment exists in a higher-dimensional space.

The Euclidean distance between the two stocks is calculated as follows:

$$\begin{aligned}\text{Euclidean Distance (A, B)} &= \sqrt{\sum (\text{ReturnA}_i - \text{ReturnB}_i)^2} = \\ &= \sqrt{(0.02 - 0.03)^2 + (-0.01 - 0.01)^2 + (0.03 - 0.02)^2 + (0.01 - 0.00)^2 + (-0.02 - (-0.01))^2} = \\ &= \sqrt{0.0001 + 0.0004 + 0.0001 + 0.0001 + 0.0001} = \\ &\approx 0.0283\end{aligned}$$

Here - ReturnA_i is the return of Stock A on day i . - ReturnB_i is the return of Stock B on day i . - And Σ is the summation over all days ($i = 1$ to 5).

We can compute Euclidean distance between Stock A and Stock B programmatically as follows:

```
[3]: # Euclidean Distance
euclidean_distance = np.sqrt(np.sum((stock_a - stock_b)**2))
print(f"Euclidean Distance: {euclidean_distance}")
```

Euclidean Distance: 0.0282842712474619

Interpretation: The Euclidean distance between Stock A and Stock B is approximately 0.0283. This indicates that the daily returns of the two stocks are relatively close to each other, suggesting some similarity in their price movements.

Manhattan Distance: Manhattan distance, also known as L1 distance or city block distance, represents the distance between two points if we could only travel along grid lines or city blocks. We can only move horizontally or vertically, not diagonally. Think of navigating a grid-like city layout. The Manhattan distance is the total distance we would travel along the streets to get from one point to another.

The Manhattan distance is calculated by summing the absolute differences of the daily returns:

$$\text{Manhattan Distance (A, B)} = \sum |\text{ReturnA}_i - \text{ReturnB}_i| = \quad (1)$$

$$= |0.02 - 0.03| + |-0.01 - 0.01| + |0.03 - 0.02| + |0.01 - 0.00| + |-0.02 - (-0.01)| = \quad (2)$$

$$= 0.01 + 0.02 + 0.01 + 0.01 + 0.01 = \quad (3)$$

$$= 0.06 \quad (4)$$

Programmatic computation of Manhattan distance between our stocks is given in the following code:

```
[4]: # Manhattan Distance
manhattan_distance = np.sum(np.abs(stock_a - stock_b))
print(f"Manhattan Distance: {manhattan_distance}")
```

Manhattan Distance: 0.06

Interpretation: The Manhattan distance between Stock A and Stock B is 0.06. This represents the total absolute difference in their daily returns over the 5-day period.

Cosine Similarity: Cosine similarity measures the cosine of the angle between two vectors. It focuses on the direction of the vectors rather than their magnitudes. Imagine two vectors originating from the same point. The cosine similarity is related to the angle between these vectors. If the vectors point in the same direction, the angle is 0 degrees, and the cosine similarity is 1 (perfect similarity). If the vectors are perpendicular, the angle is 90 degrees, and the cosine similarity is 0 (no similarity). Higher cosine similarity values indicate vectors that are closer in direction, while lower values indicate vectors that are more divergent in direction.

We can measure the similarity between the two stocks using cosine similarity formula:

$$\text{Cosine Similarity (A, B)} = \frac{A \cdot B}{\|A\| \|B\|}$$

Here $A \cdot B$ is the dot product of the daily return vectors of Stock A and Stock B. $\|A\|$ and $\|B\|$ are the magnitudes of the daily return vectors of Stock A and Stock B, respectively.

Let's complete the calculation of cosine similarity using the provided values. We first compute dot product and magnitudes:

$$A \cdot B = (0.02 * 0.03) + ((-0.01) * 0.01) + (0.03 * 0.02) + (0.01 * 0.00) + ((-0.02) * (-0.01)) = 0.0013 \quad (5)$$

$$\|A\| = \sqrt{0.02^2 + (-0.01)^2 + 0.03^2 + 0.01^2 + (-0.02)^2} \approx 0.0436 \quad (6)$$

$$\|B\| = \sqrt{0.03^2 + 0.01^2 + 0.02^2 + 0.00^2 + (-0.01)^2} \approx 0.0387 \quad (7)$$

We can now plug these in the Cosine similarity formula:

$$\text{Cosine Similarity (A, B)} = \frac{A \cdot B}{\|A\|\|B\|} = \frac{0.0013}{(0.0436 * 0.0387)} \approx 0.77$$

The following code computes cosine similarity for the given stocks:

```
[5]: # Compute dot product and magnitudes
dot_product = np.dot(stock_a, stock_b)
magnitude_a = np.linalg.norm(stock_a)
magnitude_b = np.linalg.norm(stock_b)

# Compute Cosine similarity
cosine_similarity = dot_product / (magnitude_a * magnitude_b)
print(f"Cosine Similarity: {cosine_similarity}")
```

Cosine Similarity: 0.7700535410868199

Interpretation: The cosine similarity value of 0.77 indicates a relatively high similarity between the daily returns of Stock A and Stock B. This suggests that the two stocks tend to move in similar directions, although not necessarily with the same magnitude.

Summing up: Based on the calculations we performed, we can conclude the following summary of the similarity measures for Stock A and Stock B:

- **Euclidean and Manhattan Distances:** In financial data analysis, these distances can be used to measure the overall dissimilarity between time series or vectors of financial metrics. For instance, we could use them to compare the historical returns of two stocks or to cluster stocks based on their financial characteristics. For our example both distances are relatively small (\$ \$0.0283 for Euclidean distance and \$ \$0.06 for Manhattan distance), indicating that the daily returns of Stock A and Stock B are quite similar in terms of their overall magnitude of differences. The smaller the distance, the more similar the stocks are considered to be.
- **Cosine Similarity:** Cosine similarity is often used in finance to assess the correlation or co-movement between assets. It can help identify stocks that tend to move in similar directions, even if their returns have different magnitudes. This information is valuable for portfolio diversification and risk management. For our example the cosine similarity value of 0.77

indicates a relatively high similarity in the direction of movement between the two stocks. This suggests that they tend to move in the same direction (up or down) on most days, although the magnitude of their returns might differ.

These results suggest that Stock A and Stock B exhibit a considerable degree of similarity in their price movements. They tend to move in the same direction, and the overall differences in their daily returns are relatively small.

We should remark that these similarity measures are based on a limited dataset of only 5 days. To draw more robust conclusions about the relationship between the two stocks, it's essential to analyze a larger dataset over a more extended period. Additionally, it is important to consider using other relevant metrics and techniques to gain a comprehensive understanding of their relationship.

1.1.5 1.5 Choice of similarity measure

It is important to understand the factors influencing the choice of similarity measure. The choice of similarity measure depends on the type of data, the desired interpretation of similarity and specifics of applications. Below we consider each factor in detail:

Data Type: The nature of data fundamentally guides the choice of similarity measure.

- **Numerical Data:** If data consists of continuous numerical values, we typically use distance measures like Euclidean, Manhattan, or Minkowski, or correlation coefficients like Pearson or Spearman for assessing relationships between numerical variables. These measures quantify the magnitude of differences or the strength of relationships between numerical variables.
- **Categorical Data:** When dealing with data that falls into distinct categories i.e. with categorical data represented as sets, the Jaccard Index is a common choice. It focuses on the overlap or shared characteristics between sets of categories.
- **Text Data:** Word Mover's Distance (WMD) is a way to measure the semantic similarity between text documents. WMD considers the meaning of words and the "effort" required to transform one document into another.
- **Time Series Data:** For data that changes over time, Dynamic Time Warping (DTW) is essential due to its ability to handle variations in speed and time shifts. It addresses the challenge of aligning time series that may have variations in speed or temporal shifts, enabling accurate similarity comparisons.

Desired Interpretation: The way we want to interpret the similarity results and the overall goal of analysis also play a role.

- **Distance vs. Similarity:** Whether we want to quantify dissimilarity (distance) or similarity directly depends on task's objective. Distance measures are useful for identifying differences or outliers, while similarity coefficients are better for finding similar items or grouping data points together. Distance measures result in smaller values for higher similarity, while similarity coefficients result in larger values for higher similarity. We need to choose the measure that aligns with interpretation needs.
- **Linear vs. Non-linear Relationships:** When working with numerical data, we need to consider the type of relationship we're interested in capturing. Pearson correlation is suitable for linear relationships, while Spearman Rank Correlation is more appropriate for monotonic or non-linear trends. If we're interested in linear relationships between numerical variables, Pearson correlation is appropriate. For non-linear or monotonic relationships, Spearman Rank Correlation is a better choice.

Data Characteristics: Certain characteristics of data can influence the choice of similarity measure.

- **Outliers:** If data contains outliers (extreme or unusual values), we need to consider using Manhattan distance. Manhattan distance is less sensitive to outliers compared to Euclidean distance.
- **Time Shifts (for Time Series Data):** When comparing time series, variations in speed or temporal shifts can significantly affect similarity calculations. DTW is crucial for time series data with potential time shifts or variations in speed. DTW is specifically designed to address this issue by finding the optimal alignment between time series before calculating the distance.

By carefully considering the data type, desired interpretation, data characteristics, and the specifics of applications, we can select the most appropriate similarity measure. By considering these broader contextual factors, we can make a more informed decision about the most appropriate similarity measure for specific task and data.

1.2 2 Textual Data

Alternative data refers to non-traditional data sources that are used to gain insights into investment opportunities or market trends. This type of data isn't typically found in traditional financial datasets like market prices. Alternative data is becoming increasingly important in finance as investors seek new ways to gain insights and make better investment decisions. While there are challenges associated with using alternative data, it has the potential to provide a significant advantage for those who can effectively leverage it.

What is Textual Data? Textual data, in simple terms, is data that is in the form of text. This includes things like: news articles, social media posts, earnings call transcripts, company filings, regulatory documents, customer reviews, and many other types of text-based information.

Textual data contains a wealth of information that can be valuable for investors and financial analysts. It can provide insights into:

- **Market sentiment:** By analyzing the tone and language used in news articles and social media posts, we can gauge how the market feels about specific companies, industries, or assets. This can help to predict potential market movements or identify emerging trends.
- **Company performance and strategies:** Textual data like earnings call transcripts and company filings can provide information about a company's financial performance, strategic direction, and future plans.
- **Risk assessment:** By analyzing news and social media for mentions of risks or potential threats, investors can identify emerging risks or monitor market sentiment related to specific risk factors.
- **Alternative data analysis:** Textual data is a rich source of "alternative data" that can complement traditional financial data and provide a more comprehensive view of the market or specific investments.

1.2.1 2.1 Text analysis techniques

There are several valuable approaches that can be used to analyse textual data depending on the specific task and the nature of the textual data. Here are some of the commonly used techniques:

Term Frequency-Inverse Document Frequency (TF-IDF): TF-IDF is a statistical measure that evaluates the importance of a word to a document within a collection of documents. It works by calculating two metrics: Term Frequency (TF), which measures how often a word appears in a document, and Inverse Document Frequency (IDF), which measures how rare the word is across the entire collection. By combining TF and IDF, TF-IDF highlights words that are frequent within a specific document but relatively rare across the entire corpus, effectively identifying the most distinctive and meaningful words for each document.

Word Embeddings (Word2Vec, GloVe, FastText): Word embeddings represent words as dense vectors in a high-dimensional space, where words with similar meanings are located closer to each other. This allows capturing semantic relationships between words and understanding their contextual meaning. Word embeddings are trained on large text corpora and can be used for various tasks, including sentiment analysis, document similarity, and building features for machine learning models. They enable capturing the essence of word meanings and relationships, going beyond simple word frequency counts.

Sentiment Analysis (Lexicon-based, Machine Learning-based): Sentiment analysis aims to determine the emotional tone or opinion expressed in text. It can be rule-based using sentiment lexicons, which are lists of words with associated sentiment scores, or machine learning-based using classifiers trained on labeled data. Sentiment analysis is widely used in finance to gauge market sentiment, identify customer feedback, and predict stock price movements based on news sentiment. It helps understand the overall positivity or negativity expressed in text, providing valuable insights for investment decisions.

Topic Modeling (LDA, NMF): Topic modeling uncovers hidden thematic structures in a collection of documents by identifying groups of words that frequently co-occur. This helps understand the main topics or themes discussed in the text data. Techniques like Latent Dirichlet Allocation (LDA) and Non-negative Matrix Factorization (NMF) are commonly used for topic modeling. In finance, topic modeling can be applied to identify industry trends, categorize news articles, and analyze customer reviews to understand common themes or concerns.

Named Entity Recognition (NER): Named Entity Recognition (NER) identifies and classifies named entities in text, such as people, organizations, locations, dates, and monetary values. It extracts structured information from unstructured text, making it easier to analyze and understand the relationships between different entities. In finance, NER can be used to identify key players in financial news, extract company names from regulatory filings, and understand the relationships between different entities mentioned in text.

Text Classification: Text classification assigns predefined categories or labels to text documents based on their content. It can be rule-based, using predefined rules to categorize documents, or machine learning-based, using classifiers trained on labeled data. In finance, text classification can be used for tasks like categorizing financial news articles, identifying spam or fraudulent emails, and classifying customer reviews based on sentiment. It helps automate the process of categorizing text data, making it easier to analyze and understand large volumes of textual information.

Often, a combination of these techniques is used to achieve the desired results. For example, we might use TF-IDF to identify important words, word embeddings to capture semantic relationships, and sentiment analysis to determine the overall sentiment of a document. Later in this module we delve in more details and learn applications of some of these techniques.

1.2.2 2.2 Definition of Term Frequency-Inverse Document Frequency

Term Frequency-Inverse Document Frequency (TF-IDF) is a statistical measure used to evaluate how important a word is to a document in a collection or corpus. It's widely used in information retrieval and text mining to help identify the most relevant words in a document.

- **Term Frequency (TF):**

- This component measures how frequently a term appears in a document. The basic intuition is that a word that occurs more often within a document is likely to be more important to that document's meaning. There are different ways to calculate TF, but a common one is raw frequency:

$$\text{TF}(\text{term}, \text{document}) = \frac{(\text{Number of times term appears in document})}{(\text{Total number of terms in document})}$$

- **Inverse Document Frequency (IDF):**

- IDF measures how important a term is within the whole corpus. Words that occur rarely in the corpus have a higher IDF score. IDF scales down the weight of terms that occur very frequently across the corpus and are therefore less informative about a specific document. It gives higher weight to terms that are rare. A common way to calculate IDF is:

$$\text{IDF}(\text{term}, \text{corpus}) = \log \left[\frac{(\text{Total number of documents in corpus})}{(\text{Number of documents containing the term})} \right]$$

- If a term appears in all documents, its IDF score is 0. If a term is very rare, its IDF score is high.

- **TF-IDF Calculation:**

- The TF-IDF score is simply the product of the TF and IDF scores:

$$\text{TF-IDF}(\text{term}, \text{document}, \text{corpus}) = \text{TF}(\text{term}, \text{document}) \times \text{IDF}(\text{term}, \text{corpus})$$

1.2.3 2.3 Applications of TF-IDF

While still an emerging area, the application of TF-IDF in financial investment holds potential for leveraging textual data to gain insights, manage risk, and potentially improve investment strategies. Some applications of TF-IDF specifically within the financial domain and potential use cases could be:

Sentiment Analysis for Financial Markets:

- **Gauging Market Sentiment:** TF-IDF can be used to analyze financial news articles, social media posts, earnings call transcripts, and other textual data to gauge market sentiment towards specific companies, industries, or assets. By identifying terms strongly associated with positive or negative sentiment, investors can get a sense of the overall market sentiment towards a particular investment.

- **Predicting Market Movements:** TF-IDF can contribute to building predictive models for market movements by identifying terms that are indicative of future price changes. For example, an increase in the TF-IDF scores of terms related to positive sentiment might suggest an upcoming price increase for a particular stock.
- **Example:** A hedge fund might use TF-IDF to analyze news articles and social media sentiment to predict short-term stock price movements. By identifying shifts in sentiment towards a company, they can make informed trading decisions.

Alternative Data Analysis:

- **Unstructured Data Insights:** TF-IDF can be applied to analyze unstructured data sources like earnings call transcripts, company filings, regulatory documents, and even website content to extract key information that might not be readily apparent from traditional financial data. This can help investors gain a deeper understanding of a company's performance, strategy, and future prospects.
- **Extracting Key Information:** By identifying high TF-IDF terms, investors can extract crucial information, such as mentions of new products, partnerships, or potential risks. This information can be used to make more informed investment decisions.
- **Example:** An investment firm could use TF-IDF to identify companies with innovative products or services by analyzing patent filings and research papers. By identifying terms related to innovation and technological advancements, they can identify companies with high growth potential.

Risk Management:

- **Identifying Emerging Risks:** Analyzing news and social media using TF-IDF can help identify emerging risks or potential threats to investments. By tracking the TF-IDF scores of terms related to risk factors, such as regulatory changes, economic downturns, or natural disasters, investors can get early warnings of potential market disruptions.
- **Monitoring Market Sentiment:** Tracking changes in the TF-IDF scores of specific terms related to risk factors can provide early warnings of potential market downturns. For example, an increase in the TF-IDF scores of terms related to economic uncertainty might suggest an upcoming market correction.
- **Example:** A risk manager might use TF-IDF to monitor news and social media for mentions of potential risks to their investments. By identifying emerging risks early on, they can take steps to mitigate potential losses.

Portfolio Optimization:

- **Text-Based Diversification:** TF-IDF can be used to analyze the textual descriptions of assets (e.g., company profiles, product descriptions) and diversify portfolios based on the semantic similarity or dissimilarity of their underlying businesses. This can help investors build more diversified portfolios that are less susceptible to market shocks.
- **Identifying Overlapping Exposures:** By analyzing the TF-IDF profiles of different assets, investors can identify potential overlaps in their investments. This can help them avoid overexposure to specific industries or risk factors.
- **Example:** An investment advisor might use TF-IDF to analyze the textual descriptions of different stocks in a client's portfolio to identify potential areas of overlap. This can help them diversify the portfolio and reduce risk.

Algorithmic Trading:

- **News-Based Trading:** TF-IDF can be incorporated into algorithmic trading strategies that react to news events. By identifying the sentiment and relevance of news articles, trading algorithms can make automated trading decisions based on real-time information.
- **Sentiment-Driven Trading:** Trading decisions can be automated based on real-time sentiment analysis using TF-IDF. For example, a trading algorithm might buy a stock if the sentiment towards the company suddenly becomes more positive.
- **Example:** A high-frequency trading firm might use TF-IDF to analyze news headlines and social media posts to identify trading opportunities. By reacting quickly to news events, they can potentially profit from short-term market movements.

These examples highlight the versatility and potential of TF-IDF in various financial applications. As textual data becomes increasingly important in the financial domain, TF-IDF and other text analysis techniques will continue to play a significant role in helping investors gain insights, manage risk, and improve investment strategies.

1.2.4 2.4 Benefits of TF-IDF

The benefits of using TF-IDF for text analysis, particularly in the context of finance, has many aspects that can be summarised as follows:

Enhanced Relevance and Focus on Important Words:

- **Distinctive Words:** TF-IDF highlights words that are distinctive and meaningful within a specific document relative to a larger corpus. It goes beyond simple word frequency by considering entire collection. It emphasizes terms that are frequent in a document but relatively rare across the entire collection.
- **Discriminatory Power:** By emphasizing terms that are frequent in a document but relatively rare across the entire collection, TF-IDF effectively distinguishes between common words (like “the”, “a”, “is”) and more informative words that characterize the content of a document. This helps focus on the words that truly matter for understanding the document’s meaning.
- **Improved Search Results:** In financial applications like searching for relevant news articles or research reports, TF-IDF helps retrieve documents that are more relevant to a query by prioritizing documents containing high TF-IDF terms matching the query. This ensures that the most relevant information is presented to the user.

Dimensionality Reduction and Feature Selection:

- **Feature Selection:** TF-IDF acts as a feature selection technique by assigning weights to terms. Terms with higher TF-IDF scores are considered more important features for representing the document. This helps reduce the number of features used for analysis, making the process more efficient.
- **Reduced Computational Cost:** By focusing on the most informative words, TF-IDF effectively reduces the dimensionality of the data, leading to faster processing and analysis, especially for large datasets. This is particularly important in finance, where large volumes of textual data are often analyzed.

Improved Information Retrieval and Semantic Similarity:

- **Semantic Similarity:** Documents with similar TF-IDF profiles are likely to be semantically related, even if they don’t share exact keywords. This is because TF-IDF captures the underlying meaning of documents by focusing on the most distinctive words.

- **Better Document Ranking:** Search engines and recommendation systems leverage TF-IDF to rank documents or items based on their relevance to a query or user profile. In finance, this can be used to rank news articles, research reports, or other textual data based on their relevance to a specific investment topic or company.

Versatility and Wide Applicability:

- **Wide Applicability:** TF-IDF can be applied to various text analysis tasks, including document classification, clustering, summarization, and topic modeling. This makes it a versatile tool for a wide range of financial applications.
- **Language Agnostic:** The core principles of TF-IDF are applicable to different languages, making it a versatile tool for multilingual text analysis. This is important in finance, where information is often gathered from sources in different languages.

In summary, TF-IDF is a valuable technique for enhancing the relevance of terms, reducing data dimensionality, and improving information retrieval in various text analysis applications.

1.2.5 2.5 Simplified example of applying TF-IDF

Let's illustrate the application of TF-IDF with a simplified example in the context of financial sentiment analysis.

Scenario: Suppose we have a small collection of financial news headlines related to a company called "Acme Corp.":

1. "Acme Corp. announces record profits, stock surges."
2. "Acme Corp. faces regulatory scrutiny, shares decline."
3. "Market volatility impacts Acme Corp. earnings."
4. "Acme Corp. expands into new markets, analysts optimistic."

Goal: We want to use TF-IDF to identify the most important words in each headline and understand the overall sentiment towards Acme Corp.

Steps:

Step 1. Tokenization and Preprocessing:

- Break down each headline into individual words (tokens).
- Remove stop words (common words like "the", "a", "is") and punctuation.
- Convert words to lowercase.

Step 2. Calculating Term Frequency (TF):

- For each headline, count the occurrences of each word.
- Divide the word count by the total number of words in the headline to get the TF.

Step 3. Calculating Inverse Document Frequency (IDF):

- For each word, count the number of headlines it appears in.
- Divide the total number of headlines by the number of headlines containing the word.
- Take the logarithm of the result to get the IDF.

Step 4. Calculating TF-IDF:

Multiply the TF and IDF scores for each word in each headline to get the TF-IDF score.

Illustrative Example: Let's demonstrate implementation on example Headline 1: "Acme Corp. announces record profits, stock surges."

- Tokenization and Preprocessing:
 - Tokens: ["acme", "corp", "announces", "record", "profits", "stock", "surges"]
- Calculating TF:
 - $TF("acme") = 1/7$
 - $TF("corp") = 1/7$
 - $TF("announces") = 1/7$
 - $TF("record") = 1/7$
 - $TF("profits") = 1/7$
 - $TF("stock") = 1/7$
 - $TF("surges") = 1/7$
- Calculating IDF:
 - $IDF("acme") = \log(4/4) = 0$ (appears in all headlines)
 - $IDF("corp") = \log(4/4) = 0$ (appears in all headlines)
 - $IDF("announces") = \log(4/1) = \log(4)$
 - $IDF("record") = \log(4/1) = \log(4)$
 - $IDF("profits") = \log(4/1) = \log(4)$
 - $IDF("stock") = \log(4/2) = \log(2)$
 - $IDF("surges") = \log(4/1) = \log(4)$
- Calculating TF-IDF:
 - $TF-IDF("acme") = (1/7) * 0 = 0$
 - $TF-IDF("corp") = (1/7) * 0 = 0$
 - $TF-IDF("announces") = (1/7) * \log(4)$
 - $TF-IDF("record") = (1/7) * \log(4)$
 - $TF-IDF("profits") = (1/7) * \log(4)$
 - $TF-IDF("stock") = (1/7) * \log(2)$
 - $TF-IDF("surges") = (1/7) * \log(4)$

Interpretation: Words with higher TF-IDF scores are more important for this headline. In this case, "announces", "record", "profits", and "surges" have higher TF-IDF scores, indicating their importance in conveying the positive sentiment of the headline.

We can then proceed to with applying this techniques to all eadlines. By repeating this process for all headlines, we can identify the most important words in each and understand the overall sentiment towards Acme Corp.

Python code: let's demonstrate how to apply TF-IDF using Python code with the sklearn library. The following code snippet takes a set of financial news headlines and converts them into a numerical representation (TF-IDF matrix) that captures the importance of each word in each headline. We do this by using the `TfidfVectorizer` - a powerful tool for converting a collection of raw documents into a matrix of TF-IDF features. It essentially automates the process of calculating TF-IDF scores for each word in each document:

```
[6]: # Sample headlines
headlines = [
    "Acme Corp. announces record profits, stock surges.",
    "Acme Corp. faces regulatory scrutiny, shares decline.",
    "Market volatility impacts Acme Corp. earnings.",
    "Acme Corp. expands into new markets, analysts optimistic."
]

# Create a TfidfVectorizer object
vectorizer = TfidfVectorizer(stop_words='english')

# Fit the vectorizer to the headlines
vectorizer.fit(headlines)

# Transform the headlines into a TF-IDF matrix
tfidf_matrix = vectorizer.transform(headlines)

# Get the feature names (words)
feature_names = vectorizer.get_feature_names_out()
```

After defining sample headlines in `headlines` variable we first create `TfidfVectorizer` object. `vectorizer = TfidfVectorizer(stop_words='english')` line creates an instance of the `TfidfVectorizer` class and assigns it to the variable `vectorizer`. The `stop_words='english'` argument tells the vectorizer to automatically remove common English words (like “the”, “a”, “is”) that typically don’t carry much meaning in text analysis. This helps focus on more informative words.

Then the code fit the Vectorizer. `vectorizer.fit(headlines)` line “fits” the vectorizer to the headlines data. This means the vectorizer analyzes the headlines.

Now we are ready to transform Headlines into TF-IDF Matrix. `tfidf_matrix = vectorizer.transform(headlines)` is the crucial step where the actual TF-IDF transformation happens. The transform method takes the headlines as input and converts them into a matrix representation called `tfidf_matrix`. Structure of `tfidf_matrix` has: - Rows representing the headlines; - Columns representing a unique words (tokens) from the vocabulary learned by the vectorizer; - Values that are the TF-IDF scores for each word in each headline. Higher TF-IDF scores indicate that a word is more important or distinctive within a particular headline.

After fitting the `TfidfVectorizer` to headlines and then learning a vocabulary of all the unique words present in the data, the last line of code `feature_names = vectorizer.get_feature_names_out()` retrieves the list of words (features) that were used to create the TF-IDF matrix. The list of feature names is then assigned to the variable `feature_names`.

Finally, we can print the list of feature names (words) that were extracted from headlines and used to create the TF-IDF matrix:

```
[7]: # Print the feature names
print(feature_names)
```

```
['acme' 'analysts' 'announces' 'corp' 'decline' 'earnings' 'expands']
```

```
'faces' 'impacts' 'market' 'markets' 'new' 'optimistic' 'profits'
'record' 'regulatory' 'scrutiny' 'shares' 'stock' 'surges' 'volatility']
```

This output represents the vocabulary of unique words that the `TfidfVectorizer` extracted from headlines after preprocessing (removing stop words). The words in the list are usually sorted alphabetically. Each word in this list corresponds to a column in the TF-IDF matrix (`tfidf_matrix`). The values in that column represent the TF-IDF scores for that particular word in each headline.

We can use `feature_names` to access specific words. When we want to analyze the TF-IDF score for a particular word, we can use the `feature_names` list to find its index. The following code focuses on accessing TF-IDF scores for a specific headline and word. This code snippet identifies the specific headline (first headline) and word (“profits”) that we are interested in. Then it finds the corresponding row and column in the TF-IDF matrix, retrieves the TF-IDF score from that location in the matrix and prints the score:

```
[8]: # Accessing TF-IDF scores for a specific headline and word
headline_index = 0 # Index of the first headline
word_index = feature_names.tolist().index('profits') # Get the index of the
↳ word "profits"

# Find TF-IDF score and print
tfidf_score = tfidf_matrix[headline_index, word_index]
print(f"TF-IDF score for 'profits' in the first headline: {tfidf_score}")
```

TF-IDF score for 'profits' in the first headline: 0.42468159315633897

The TF-IDF matrix is a core component of TF-IDF analysis, as it stores the calculated TF-IDF scores for each word in each document. Let’s now see a numerical representation of the TF-IDF matrix. The following line of code displays the numerical representation of the TF-IDF matrix:

```
[9]: # Print the TF-IDF matrix
print(tfidf_matrix.toarray())
```

```
[[0.22161647 0.          0.42468159 0.22161647 0.          0.
  0.          0.          0.          0.          0.          0.
  0.          0.42468159 0.42468159 0.          0.          0.
  0.42468159 0.42468159 0.          ]
 [0.22161647 0.          0.          0.22161647 0.42468159 0.
  0.          0.42468159 0.          0.          0.          0.
  0.          0.          0.          0.42468159 0.42468159 0.42468159
  0.          0.          0.          ]
 [0.24478737 0.          0.          0.24478737 0.          0.46908376
  0.          0.          0.46908376 0.46908376 0.          0.
  0.          0.          0.          0.          0.          0.
  0.          0.          0.46908376]
 [0.22161647 0.42468159 0.          0.22161647 0.          0.
  0.42468159 0.          0.          0.          0.42468159 0.42468159
  0.42468159 0.          0.          0.          0.          0.
  0.          0.          0.          ]]
```

While `print(tfidf_matrix.toarray())` gives the numerical representation, it lacks context.

Here's how we can improve it and make the output of the TF-IDF matrix more readable by adding row and column headings. The most convenient way to add headings is to use the Pandas library to create a DataFrame. This allows to label rows and columns, making the output much clearer.

```
[10]: # Create a list of generic headline labels
headline_labels = ["Headline1", "Headline2", "Headline3", "Headline4"]

# Create a Pandas DataFrame
pd.DataFrame(tfidf_matrix.toarray(), columns=feature_names,
             index=headline_labels)
```

```
[10]:
```

	acme	analysts	announces	corp	decline	earnings	
Headline1	0.221616	0.000000	0.424682	0.221616	0.000000	0.000000	\
Headline2	0.221616	0.000000	0.000000	0.221616	0.424682	0.000000	
Headline3	0.244787	0.000000	0.000000	0.244787	0.000000	0.469084	
Headline4	0.221616	0.424682	0.000000	0.221616	0.000000	0.000000	

	expands	faces	impacts	market	...	new	optimistic	
Headline1	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	\
Headline2	0.000000	0.424682	0.000000	0.000000	...	0.000000	0.000000	
Headline3	0.000000	0.000000	0.469084	0.469084	...	0.000000	0.000000	
Headline4	0.424682	0.000000	0.000000	0.000000	...	0.424682	0.424682	

	profits	record	regulatory	scrutiny	shares	stock	
Headline1	0.424682	0.424682	0.000000	0.000000	0.000000	0.424682	\
Headline2	0.000000	0.000000	0.424682	0.424682	0.424682	0.000000	
Headline3	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
Headline4	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	

	surges	volatility
Headline1	0.424682	0.000000
Headline2	0.000000	0.000000
Headline3	0.000000	0.469084
Headline4	0.000000	0.000000

[4 rows x 21 columns]

Now, instead of just seeing an array of numbers, we have a nicely formatted table with the headlines as row labels and the words as column labels. Each cell in the table contains the TF-IDF score for that word in that headline.

This simplified example demonstrates the basic steps involved in applying TF-IDF. In real-world applications, we might need to perform more advanced preprocessing, use different parameters for the `TfidfVectorizer`, and apply more sophisticated analysis techniques to extract meaningful insights from financial text data.

1.3 Conclusion

In this lesson we primarily focused on similarity measures and their applications in financial markets. We introduced the concept of similarity measures, explaining how they are used to quantify the likeness between data objects. We learned that there are two main categories of similarity measures: Distance Measures and Similarity Coefficients:.

We then shifted focus on to Term Frequency-Inverse Document Frequency (TF-IDF), a statistical measure used to evaluate the importance of words in a document within a collection of documents. We learned about how TF-IDF works, its benefits, and its applications in financial analysis. These applications include sentiment analysis for financial markets, alternative data analysis, risk management, algorithmic trading, and portfolio optimization.

In the next lesson we try to discover how to apply these techniques in practice.

References

- Daniel Jurafsky and James H. Martin. (2024). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition. Online manuscript released August 20, 2024. <https://web.stanford.edu/~jurafsky/slp3>
- Wikipedia Contributors (2024). Similarity measure. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Similarity_measure

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Preis, T., Moat, H.S. and Stanley, H.E. (2013). Quantifying Trading Behavior in Financial Markets Using Google Trends. Scientific Reports, 3(1). doi:<https://doi.org/10.1038/srep01684>.

- **Required pages:** The entire content
- **Estimated reading time:** 30 minutes

2. Challet, D. and Bel, A. (2014). Predicting financial markets with Google Trends and not so random keywords. [online] arXiv.org. Available at: <https://doi.org/10.48550/arXiv.1307.4643>

- **Required pages:** The entire content
- **Estimated reading time:** 25 minutes

3. Aslanidis, N., Bariviera, A.F. and López, Ó.G. (2022). The link between cryptocurrencies and Google Trends attention. Finance Research Letters, [online] 47, p.102654. doi:<https://doi.org/10.1016/j.frl.2021.102654>.

- **Required pages:** The entire content
- **Estimated reading time:** 30 minutes

Lesson 2 - Search Engine Data

December 25, 2025

MODULE 4 | LESSON 2

1 Search Engine Data

Reading Time	60 minutes
Prior Knowledge	Basic Python, Basic concepts of Natural Language Processing, Financial Markets
Keywords	Machine Learning, Google Trends, Pytrends, Sentiment Analysis, TF-IDF, LSA (Latent Semantic Analysis), NLP (Natural Language Processing)

In this lesson we explore how to leverage data science and machine learning for financial analysis, with a focus on analyzing search terms from Google Trends data. It covers techniques like TF-IDF and LSA to extract insights from text, and demonstrates how to gather and analyze Google Trends data using Python and the `pytrends` library. The notebook also discusses the application of these techniques for sentiment analysis, risk management, and portfolio optimization.

```
[1]: # Load libraries
import matplotlib.pyplot as plt
import nltk
import pandas as pd
import plotly.graph_objects as go
import re
import time

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from pytrends.request import TrendReq
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Download 'punkt' resource
```

```
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

[1]: True

1.1 1 What is Search Engine Data

Search engine data refers to the information collected and generated by search engines as users interact with them. This data encompasses various aspects of user behavior, preferences, and interactions within the search engine ecosystem. Search engine data is a valuable alternative data source that can complement traditional financial data and provide a unique perspective on market trends and investor behavior.

When it comes to financial engineering, using search engine data is a more specialized area. We generally have two options: Using APIs from major search engines such as Google Trends API or Bing API; or Alternative data providers: Several companies specialize in providing alternative data for financial analysis, including search engine data.

In this lesson we will focus on using Google Trends data. Here are some suggestions on how Google Trends data can be used for the financial engineering tasks:

- **Sentiment analysis:** Google Trends data can be used to measure the sentiment of the market towards a particular security or asset. This can be done by tracking the search volume for keywords related to the security or asset. For example, if the search volume for “Tesla stock price” is increasing, this could be a sign that the market is becoming more bullish on Tesla.
- **Predictive modeling:** Google Trends data can be used to build predictive models for financial markets. This can be done by using machine learning algorithms to train models on historical Google Trends data and other relevant data sources. For example, a model could be trained to predict the price of a stock based on the search volume for keywords related to the stock.
- **Risk management:** Google Trends data can be used to identify and manage risks in financial markets. This can be done by tracking the search volume for keywords related to risk factors, such as “economic recession” or “market volatility.” For example, if the search volume for “economic recession” is increasing, this could be a sign that the market is becoming more concerned about the possibility of a recession.
- **Algorithmic trading:** Google Trends data can be used to develop algorithmic trading strategies. This can be done by using Google Trends data to identify trading opportunities. For example, a trading strategy could be developed to buy a stock when the search volume for keywords related to the stock is increasing.

Search engine data can be valuable for investors, and TF-IDF can play a role in its analysis. Here’s how:

How Search Engine Data Helps Investors:

- **Gauging Public Interest:** Search engine data, like Google Trends, reveals the relative popularity of search terms over time. This can indicate public interest in specific companies, products, industries, or economic trends.
- **Identifying Emerging Trends:** Increases in search volume for specific terms can signal emerging trends or shifts in consumer behavior, providing early investment opportunities.
- **Sentiment Analysis:** Analyzing search queries related to a company or asset can provide insights into public sentiment, which can influence investment decisions.
- **Predictive Modeling:** Search engine data can be combined with other data sources to build predictive models for market movements or company performance.

Role of TF-IDF in Analyzing Search Engine Data:

- **Keyword Analysis:** TF-IDF can be applied to analyze the search queries themselves, identifying the most relevant and informative terms related to an investment topic.
- **Document Analysis:** TF-IDF can be used to analyze the content of web pages returned in search results, helping investors understand the context and sentiment surrounding a particular search term.
- **Trend Identification:** Tracking changes in the TF-IDF scores of specific terms over time can help identify emerging trends or shifts in public interest.
- **Examples:**
 - An investor might analyze Google Trends data for search terms related to electric vehicles to assess the growing interest in this industry and identify potential investment opportunities.
 - A hedge fund could use TF-IDF to analyze news articles and social media posts related to a company to gauge public sentiment and predict potential stock price movements.

Benefits for Investors:

- **Early Signals:** Search engine data can provide early signals of market trends or changes in investor sentiment.
- **Alternative Data Source:** It offers an alternative data source that can complement traditional financial data.
- **Improved Decision-Making:** By incorporating search engine data analysis, investors can make more informed investment decisions.

Considerations:

- **Data Quality:** Search engine data can be noisy and may not always accurately reflect market sentiment.
- **Privacy Concerns:** Ethical considerations and data privacy regulations need to be taken into account when using search engine data.

Overall, search engine data can be a valuable tool for investors, and TF-IDF can be effectively used to analyze this data and extract meaningful insights for investment decisions.

1.1.1 1.1 Introduction to Google Trends data

It is important to note that Google Trends data is just one of many data sources that can be used for financial engineering tasks. It is important to use a variety of data sources to get a complete

picture of the market.

In this lesson, we will explore how to use Python to fetch and analyze data from Google Trends, a powerful tool that reveals how often a particular search term is entered relative to the total search volume across various regions of the world and in various languages. We will use the `pytrends` library, a Python interface for Google Trends data. This library allows us to automate the downloading of reports from Google Trends. We will learn how to fetch the interest over time for a specific search term, how to analyze this data using Python, and how to visualize findings. It provides an interface to Google Trends, allowing to:

- Fetch Interest Over Time: See how search interest for specific keywords has changed over a period.
- Analyze Interest by Region: Discover where in the world people are searching for particular terms.
- Find Related Queries: Identify top and rising search queries related to our keywords.

By the end of this section, we will have a solid understanding of how to leverage Python and Google Trends data to uncover insights about search trends. This knowledge can be particularly useful in fields like market research, where understanding trends can provide valuable insights into consumer behavior.

1.1.2 1.2 Fetching and Comparing Interest Over Time

Now let's define the search terms that we want to compare. In this case, we'll compare "AAPL", "TSLA", and "NVDA". We fetch the interest over last 12 month period for these search terms from Google Trends and visualize the data. The following code snippet initializes an instance of the `TrendReq` class from the `pytrends` library and defines a list of keywords and a timeframe. This sets up the necessary objects to interact with Google Trends and retrieve data. Then the code builds the payload for the Google Trends request using the defined keywords and timeframe, and then retrieves the interest over time data. The `interest_over_time_df` DataFrame would then contain the Google Trends data for the specified keywords and timeframe:

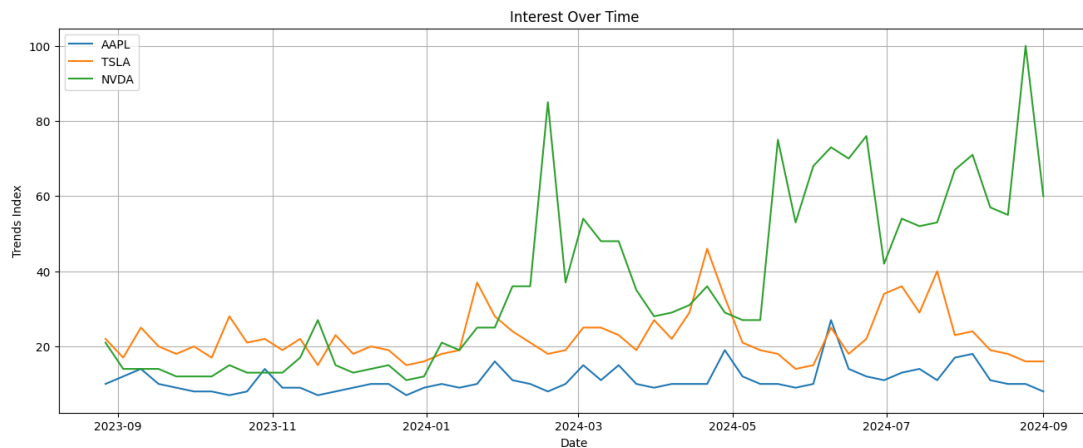
```
[2]: # Initialize an instance of the TrendReq class with US region and 360 timezone
pytrends = TrendReq(hl='en-US', tz=360)

# Define the list of keywords and timeframe
kw_list = ["AAPL", "TSLA", "NVDA"]
timeframe = '2023-09-01 2024-09-01' # for latest 12 months replace with 'today_
↳ 12-m'

# Build the payload and get the interest over time data
pytrends.build_payload(kw_list, timeframe=timeframe)
interest_over_time_df = pytrends.interest_over_time()

# Create a figure and axes and plot the data
plt.figure(figsize=(16, 6))
for term in kw_list:
    plt.plot(interest_over_time_df.index, interest_over_time_df[term], label=term)
```

```
plt.xlabel('Date')
plt.ylabel('Trends Index')
plt.title('Interest Over Time')
plt.legend(loc='upper left')
plt.grid(True)
plt.show()
```



Now that we have the plot, here are some key things to pay attention to and analyze:

Overall Trends: - Upward/Downward: Are the trends generally increasing, decreasing, or remaining stable over the year? This can indicate growing or waning interest in the companies. - Seasonality: Do we notice any recurring patterns or spikes at certain times of the year? This could be related to product releases, financial reports, or other events.

Relative Popularity: - Comparison: Which keyword shows the highest overall interest? Which one shows the lowest? This can give us an idea of the relative popularity of these companies in terms of search volume. - Crossovers: Are there points where the trends for different keywords intersect? This could signal shifts in public attention or perception.

Volatility: - Fluctuations: How much do the trends fluctuate over time? Large swings might suggest sensitivity to news or market events. - Outliers: Are there any significant spikes or dips that stand out? Try to investigate what might have caused them (e.g., news announcements, product launches, controversies).

Correlation: - Relationship: Do the trends for different keywords seem to move together or independently? This can indicate whether they are influenced by similar factors or not.

By carefully examining these aspects of the plot, we can gain insights into the search behavior related to these companies and potentially draw conclusions about their public perception and market performance.

For our example specifically, peaks and troughs can be observed, with NVIDIA (NVDA) experiencing a significant peak around March 2024, nearly reaching a trend index of 80, and then again over most recent period by end of August 2024. Such high index could coincide with significant events

related to NVIDIA, such as product launches, earnings reports, or market developments. In fact, in March 2024, NVIDIA made significant announcements related to its next-generation GPU architecture called Blackwell. This architecture boasts an impressive 208 billion transistors and is designed to run real-time generative AI models. The anticipation around Blackwell likely contributed to the increased interest in NVIDIA during that time. Additionally, in August 2024, NVIDIA continued to unveil innovations related to AI and data center performance at the Hot Chips conference, further fueling interest in the company. These events and technological advancements likely explain the observed spikes in interest for NVIDIA during those months.

Apple (AAPL) and Tesla (TSLA) exhibit varying trends over the same period. While both companies generally maintain interest, their fluctuations are less pronounced compared to NVIDIA. Analyzing news or corporate announcements during these timeframes might shed light on the reasons behind these trends.

We also need to consider exploring correlations between the interest trends in these companies and broader market indices (e.g., S&P 500). If there's a strong positive correlation, it could indicate that overall market sentiment influences interest in specific stocks.

We also need to observe any recurring patterns across different months or quarters. Seasonal factors (e.g., holiday seasons, earnings seasons) might impact investor sentiment and interest levels.

- **TSLA Peaks:** Tesla (TSLA) usually experiences noticeable peaks around April and July each year. These could align with quarterly earnings reports or significant announcements related to Tesla's products or business developments.
- **AAPL Trends:** Apple (AAPL) usually shows a consistent upward trend around September and January. These months often coincide with new product launches (e.g., iPhone releases) and holiday seasons.
- **NVDA Fluctuations:** NVIDIA (NVDA) exhibits less predictable patterns, but there are fluctuations around March and August. Investigate whether industry-specific events or conferences occur during these months.

1.1.3 1.3 Analyzing Interest by other metrics

Google Trends also allow to fetch other details in relation to our search query:

- **Interest by subregion** lets us see how interest in a particular search term varies across different geographical areas within a larger region.
- **Related topics** are broader subjects or concepts that are associated with our keyword, and which were also searched by other users.
- **Related queries** are specific search terms that people have used in conjunction with our keyword. These are actual queries entered into the search engine by other users during the same time period.

Analyzing Interest by Region Now, let's fetch the interest by region for a search term NVDA and identify the top 10 regions with the highest interest:

```
[3]: # Initialize an instance of the TrendReq class and build the payload
pytrends = TrendReq hl='en-US', tz=360)
pytrends.build_payload(["NVDA"], timeframe='2023-09-01 2024-09-01')
```

```
# Get the interest by region data
interest_by_region_df = pytrends.interest_by_region()
interest_by_region_df.sort_values(by='NVDA', ascending=False).head(10)
```

```
[3]:
```

	NVDA
geoName	
Hong Kong	100
Israel	80
Taiwan	77
United States	67
Canada	62
Singapore	62
China	34
Switzerland	16
South Korea	13
United Arab Emirates	11

This output shows the top 10 regions with the highest search interest for “NVDA” during the specified period (‘2023-09-01 2024-09-01’). The values represent the relative popularity of the search term in each region, where 100 is the highest value.

For example, Hong Kong has the highest search interest for “NVDA” with a value of 100, followed by Taiwan and Israel with a value of 79 each.

This data can be useful for businesses to understand the geographic distribution of interest in their products or services. It can also be used for market research and to identify potential target markets. Please note that further analysis and correlation with other data sources would be needed to draw definitive conclusions.

Related Queries and Topics Top and rising related topics and queries can be extremely useful for financial engineering tasks because they provide insights into the collective “mind of the market” and reveal trends that might not be obvious from traditional financial data.

Related Topics are broader themes associated with NVDA. This can help to identify: - Industry trends: Are there broader trends in gaming, AI, or data centers that are influencing interest in NVDA? - Investment themes: Are there specific investment themes (e.g., growth stocks, tech stocks) that are driving interest in NVDA?

Related Queries are other terms people search for in conjunction with NVDA keyword. This can reveal: - Emerging trends: Are there new or rising keywords that indicate shifting interest or new areas of focus for NVDA? - Competitor analysis: Are people searching for NVDA in comparison to its competitors? - Public perception: What are the main concerns or interests people have regarding NVDA?

Both Related Topics and Related Queries can be filtered to get Top and Rising terms for each:

- **Top** - these are most popular search topics/queries. Scoring is on a relative scale from 100 as the most commonly searched topic/querie, 50 for query searched half as often, and so on.
- **Rising** - these are topics/queries with the biggest increase in search frequency since the last time period. Results marked “Breakout” had a tremendous increase, probably because these topics/queries are new and had few (if any) prior searches.

In this lesson we will focus on Related Queries. Both top and rising related queries offer valuable insights, but they serve different purposes. The best choice depends on specific goals and the type of analysis we are conducting. From a financial engineering perspective, the choice between top and rising related queries still depends on our specific goals, but here's a more tailored recommendation:

- **For Algorithmic Trading and Sentiment Analysis:** Rising queries are generally more valuable. They can provide real-time signals for sentiment-based trading strategies. We could build a system that automatically adjusts portfolio positions or generates trading signals based on changes in the sentiment expressed in rising queries. For example if we observe a sudden surge in negative sentiment related to a company we hold in our portfolio, algorithm could automatically reduce exposure to that stock.
- **For Risk Management:** Rising queries are crucial for early warning signs. A spike in searches for terms related to lawsuits, regulatory investigations, or product recalls could indicate potential risks for a company or industry. We can integrate rising query analysis into risk management system to identify and assess potential threats proactively.
- **For Portfolio Optimization and Asset Allocation:** Both top and rising queries can be useful.
 - Top queries: Can help to understand the main themes and drivers of different assets or industries, allowing to diversify portfolio accordingly.
 - Rising queries: Can help to identify emerging themes or potential risks that might warrant adjustments to portfolio.
- **For Quantitative Research and Modeling:** Both types of queries can be valuable as alternative data sources.
 - Rising queries: Can be used to create sentiment indicators or event detection signals that can be incorporated into quantitative models.
 - Top queries: Can be used to understand the relationships between different assets or industries based on the themes and concerns expressed in search queries.

Overall, while both have their uses, rising queries tend to be more valuable for financial engineering applications that require real-time insights and a focus on short-term trends and sentiment shifts. Top queries provide a broader understanding of market perception, which can be useful for longer-term strategies and fundamental analysis. Remember to consider the challenges of noise, data quality, and the need for sophisticated analysis techniques when working with both types of queries.

Please also note that narrowing the time period for fetching insights from rising related queries can be very beneficial, especially when our goal is to detect short-term trends or capture immediate market reactions. Shorter timeframe generally provides a good balance between sensitivity, data availability, and actionability. We can experiment with different timeframes to find what works best for our specific needs and the volatility of the assets we are analyzing.

Let's now fetch the top and rising related queries for 'NVDA' search term for the period from 2024-05-01 to 2024-09-01 and analyze the results. This time we manually downloaded the data in CSV format and saved locally as 'NVDA-relatedQueries.csv'. CSV file from Google Trends containing related queries typically presents the data in a stacked format: The "Rising" queries data is typically stacked below the "Top" queries data in the same CSV file. There might be a blank row or a visual separator (like a line of dashes) to distinguish the two sections. The code below addresses the specific structure of the Google Trends CSV file, particularly the stacking of "Top"

and “Rising” queries. First we read CV file into DataFrame. Then find the start and end rows for each section and finally extract each section with simultaneously splitting column on comma and formatting values:

```
[4]: # Read the CSV file
df = pd.read_csv('NVDA-relatedQueries.csv', header=None, delimiter='\t')

# Find the start and end rows for each section
top_start = df[df.iloc[:, 0] == 'TOP'].index[0] + 1
top_end = df[df.iloc[:, 0] == 'RISING'].index[0] - 1
rising_start = df[df.iloc[:, 0] == 'RISING'].index[0] + 1
rising_end = df.shape[0] # End of the DataFrame
```

```
[5]: # Extract top queries
queries_top = df.iloc[top_start:top_end].copy()
queries_top = queries_top.rename(columns={queries_top.columns[0]:
    ↪ 'top_queries'})

# Split the 'top_queries' column and convert 'value' to numeric
queries_top[['top_query', 'value']] = queries_top['top_queries'].str.split(',',
    ↪ expand=True)
queries_top = queries_top[['top_query', 'value']]
queries_top.loc[:, 'value'] = pd.to_numeric(queries_top['value'])
queries_top
```

```
[5]:
```

	top_query	value
3	stock nvda	100
4	nvda price	23
5	stock price nvda	20
6	tsla	7
7	amd	7
8	amd stock	7
9	aapl	7
10	tsla stock	6
11	aapl stock	6
12	amzn	5
13	msft	5
14	nvda earnings	5
15	smci	5
16	amzn stock	4
17	msft stock	4
18	tesla stock	4
19	meta	4
20	smci stock	4
21	meta stock	4
22	nvda split	3
23	gme	3

24	qqq	3
25	spy	3
26	pltr	3

```
[6]: # Extract rising queries
queries_rising = df.iloc[rising_start:rising_end].copy()
queries_rising = queries_rising.rename(columns={queries_rising.columns[0]:
↳ 'rising_queries'})

# Split the 'rising_queries' column aon the first comma
queries_rising[['rising_query', 'value']] = queries_rising['rising_queries'].
↳ str.split(',', n=1, expand=True)
queries_rising = queries_rising[['rising_query', 'value']]

# Clean the 'value' column
queries_rising['value'] = queries_rising['value'].str.replace(r'[^0-9.-]', '',
↳ regex=True) # Keep only numbers, dots, and minus signs
queries_rising.loc[:, 'value'] = pd.to_numeric(queries_rising['value'],
↳ errors='coerce') # Convert to numeric

# Display 'rising_queries'
queries_rising
```

```
[6]:      rising_query value
29      ffie stock  3200
30      ffie      3150
31      serv stock  2800
32      asts      750
33      asts stock   700
34      nvda split date  700
35 nvda stock split date  550
36      gme stock   550
37      gme      550
38  gamestop stock   400
39      hims stock   250
40      cmg stock   250
41      cmg stock price  200
42  dell stock price   190
43      dell stock   150
44      chipotle stock   150
45      broadcom stock   140
46      djt stock price   130
47      cava stock   110
48      djt stock   110
49      nvda split   100
50      avgo stock   100
51  crowdstrike stock   100
```


52	<code>nvax stock</code>	100
53	<code>nke stock</code>	90

The rising queries for “NVDA” provides some interesting insights, particularly regarding the upcoming earnings announcement. Here’s what we can gather:

- **Earnings Dominate:** The overwhelming focus is on NVDA’s earnings. Terms related to earnings are all experiencing massive increases in search interest. This signifies a high level of anticipation and interest in the company’s financial results.
- **Real-Time Information:** The “Breakout” trend for “nvda earnings time today” suggests that people are actively seeking real-time information about the earnings announcement, possibly during the day of the release.
- **Global Interest:** The presence of Japanese and Chinese characters indicates that interest in NVDA’s earnings extends beyond English-speaking regions, highlighting the company’s global reach.
- **Investor Focus:** Terms like “nvda investor relations” and “when does nvda report earnings” suggest that investors are actively seeking information about the earnings release and possibly preparing for potential market reactions.
- **Financials and Nasdaq:** The rise of “nasdaq nvda financials” suggests that people are looking for detailed financial information about NVDA, possibly on the Nasdaq website or other financial platforms.
- **Other Stocks and Potential Noise:** While earnings-related terms dominate, there are some outliers like “lunr stock,” “pdd stock,” “crm stock,” “chwy stock,” and “afm stock” These might be related to other companies or market trends and could potentially be noise in the context of analyzing NVDA. Further investigation would be needed to understand their relevance.

1.1.4 1.4 Considerations and Limitations when using Google Trends data

When using Google Trends data for financial analysis, we should be aware of its limitations. While Google Trends data can be insightful, it’s crucial to understand its limitations regarding data quality. Here are some key aspects to keep in mind:

- **Sampling:** Google Trends data is based on a sample of searches, not the entire search volume and the exact sampling methodology isn’t publicly disclosed. This means there’s inherent sampling error, and the data might not perfectly represent the entire population of searches. This means there might be some noise and the data may not perfectly reflect the actual search interest.
- **Data Updates:** Google Trends data might be updated or revised over time as Google refines its algorithms and data processing techniques. This means that historical data you accessed previously could potentially change slightly in the future.
- **Normalization:** Google Trends data is normalized, meaning the values are relative to the highest point in the timeframe you select. The data is normalized to a scale of 0 to 100. A value of 100 represents the peak popularity, while other values are scaled proportionally. This means the values represent the relative popularity of a search term compared to the highest point in the selected timeframe. While normalization helps in comparing trends, it can also mask the absolute search volume, which might be relevant in some cases.

- **Geographic Granularity:** The level of geographic detail available in Google Trends data varies depending on the search term and the time period. For some terms or regions, the data might be more granular, while for others, it might be more aggregated. This can affect the accuracy of regional analysis.
- **External Factors:** Search behavior can be influenced by various external factors, such as media coverage, news events, seasonality, and even the popularity of other search terms. These factors can introduce noise and bias into the data, making it challenging to isolate the true underlying trends.
- **Privacy:** Google Trends data is anonymized and aggregated, but it's still important to be mindful of privacy considerations, especially when dealing with sensitive topics.

By understanding these data quality considerations and taking appropriate precautions, we can use Google Trends data more effectively and draw more reliable conclusions for financial analysis. Remember that context is key. We need to consider the context surrounding the data, such as potential external factors that might be influencing search behavior.

1.2 2 Latent Semantic Analysis

Up until now we explored Google Trends data and explored simple examples of using `pytrends` to gather data. While `pytrends` itself provides valuable insights into search trends, incorporating machine learning techniques like Latent Semantic Analysis (LSA) can offer several advantages.

Before introducing LSA, we need to understand some terminology. In the context of LSA, a “**document**” refers to a unit of text that we want to analyze. It can be anything from a single sentence to a whole article, a book, or even a collection of related texts. The key is that it represents a distinct piece of information that we want to compare and analyze in relation to other documents. Here are some examples of what could be considered a “document” in different LSA applications:

- **Topic Modeling:** Each document could be a news article, a blog post, or a social media post. LSA can help identify the main topics discussed across these documents.
- **Information Retrieval:** Each document could be a web page, a research paper, or a product description. LSA can help find documents that are semantically similar to a given query.
- **Document Classification:** Each document could be an email, a customer review, or a legal contract. LSA can help categorize these documents based on their content.

Latent Semantic Analysis (LSA) is a technique in natural language processing that analyzes relationships between a set of documents and the terms they contain by producing a set of concepts related to the documents and terms. LSA assumes that words that are close in meaning will occur in similar pieces of text. Incorporating machine learning techniques like LSA can offer several advantages:

- **Uncovering Hidden Relationships:** LSA can reveal hidden relationships between terms and concepts that might not be apparent from simply looking at raw search data. This can help us understand the underlying semantic structure of the search queries related to our documents.
- **Dimensionality Reduction:** LSA can reduce the dimensionality of our data by representing it in a lower-dimensional concept space. This can make it easier to analyze and visualize the data, especially when dealing with a large number of terms.

- **Improved Search and Recommendation:** By understanding the relationships between terms and concepts, we can build more intelligent search engines and recommendation systems. For example, we could suggest related searches or recommend documents based on semantic similarity rather than just keyword matching.
- **Topic Modeling:** LSA can be used for topic modeling, which allows us to discover the main topics or themes present in a set of documents or search queries. This can help us understand the broader context of the information and potentially categorize or cluster documents more effectively.

In a nutshell, LSA is used to analyze textual data and is a method for extracting and representing the contextual-usage meaning of words by statistical computations applied to a large corpus of text. LSA is a powerful tool for analyzing textual data, but it has some limitations. For example, it is not very good at handling polysemy (words with multiple meanings) or synonymy (different words with the same meaning).

In this lesson we will explore basic example of how we could use LSA to analyze related search terms and improve our understanding of user search behavior. The choice of what constitutes a document depends on our specific application and the kind of analysis we want to perform. It's important to ensure that our documents are meaningful and relevant to the task at hand. A collection of documents is collectively called as “**term-document matrix**”.

Financial engineers might approach document composition for LSA or similar techniques in different ways:

- **Domain Expertise:** Financial engineers often have deep domain expertise and carefully select keywords and phrases based on their understanding of financial markets, specific assets, and relevant events.
- **Data Sources:** They may use a wider range of data sources beyond just Google Trends, such as News articles and sentiment analysis, Social media data, Company filings and earnings call transcripts, Proprietary datasets.
- **Sophisticated Preprocessing:** They might employ more advanced natural language processing techniques for preprocessing, including Named entity recognition (identifying companies, people, locations), Sentiment analysis (assigning sentiment scores to text), Topic modeling (identifying underlying topics in documents).
- **Ensemble Methods:** They may combine LSA with other machine learning techniques or use ensemble methods to improve the accuracy and robustness of their models.
- **Backtesting and Validation:** Financial engineers rigorously backtest and validate their models using historical data to ensure they are reliable and can potentially generate profitable trading signals or investment strategies.

For our example we will create term-document matrix with the following set of documents:

Document 1: “Risk management in derivatives trading”,
 Document 2: “Portfolio optimization techniques and strategies”,
 Document 3: “Algorithmic trading and its impact on market efficiency”,
 Document 4: “Quantitative methods for financial modeling”,
 Document 5: “High-frequency trading and liquidity provision”,
 Document 6: “Integrating Derivatives into Portfolio Optimization Strategies”,

Document 7: “High-Frequency Trading Strategies: Algorithmic Approaches and Market Impact”,
Document 8: “Quantitative Methods for Algorithmic Trading and Portfolio Optimization”

Before proceeding with code, let’s observe that documents in our term-document matrix contain such words as “in”, “and”, “for”, “on”. These words are called **stop words**. They are common words that generally don’t carry much specific meaning in the context of text analysis. It’s often beneficial to remove stop words from text data before creating the term-document matrix. This can help to:

- **Reduce noise:** Stop words can add noise to our data and obscure the more important terms that convey the main topics or themes.
- **Improve efficiency:** Removing stop words can reduce the size of our matrix, making computations faster and more efficient.
- **Focus on relevant terms:** By eliminating common words, we can focus on the more meaningful terms that are specific to our documents and application.

We will also apply lemmatization technique. **Stemming and lemmatisation** are two text preprocessing techniques used in Natural Language Processing (NLP) that help to reduce words to their base form. - **Stemming** reduces words to their stem, which are not necessarily real words, by removing prefixes and suffixes. For example “playing” would become “play”, “studies” would become “studi”, and “happily” would become “happi”. - **Lemmatization** reduce words to their dictionary form or base form known as lemma. For example “playing” would become “play”, “studies” would become “study”, and “better” would become “good”.

In our example we consider lemmatization: Given the specific and potentially nuanced language of finance, lemmatization is likely to be more beneficial than stemming. It can help group different forms of words (e.g., “analyze,” “analyzing,” “analysis”) while preserving their core meaning.

Now that we made clear of all basic terminology and LSA definition, let’s proceed with preparing our data. The following Python code performs exactly that:

```
[7]: # Sample documents
documents = [
    "Risk management in derivatives trading",
    "Portfolio optimization techniques and strategies",
    "Algorithmic trading and its impact on market efficiency",
    "Quantitative methods for financial modeling",
    "High-frequency trading and liquidity provision",
    "Integrating Derivatives into Portfolio Optimization Strategies",
    "High-Frequency Trading Strategies: Algorithmic Approaches and Market_
↵Impact",
    "Quantitative Methods for Algorithmic Trading and Portfolio Optimization"
]

# Download stopwords and WordNet resources and then create a Lemmatizer Object
nltk.download('stopwords')
nltk.download('wordnet')
```

```

nltk.download('punkt')
lemmatizer = WordNetLemmatizer()

# construct text preprocessing function
def preprocess_text(text):
    text = re.sub(r'[\w\s-]', '', text) # Remove punctuation except hyphens
    tokens = nltk.word_tokenize(text.lower())
    tokens = [lemmatizer.lemmatize(token) for token in tokens] # Lemmatize
    stop_words = set(stopwords.words('english')) # define stop_words as a set
    ↪ of english stop words
    tokens = [token for token in tokens if token not in stop_words] # Remove
    ↪ stop words
    return tokens

# Create a CountVectorizer object
vectorizer = CountVectorizer(tokenizer=preprocess_text, token_pattern=None)

# Create the term-document matrix, then convert to a pandas DataFrame
term_document_matrix = vectorizer.fit_transform(documents)
df = pd.DataFrame(term_document_matrix.toarray(), columns = vectorizer.
    ↪ get_feature_names_out())
term_document_matrix = df.T
print(term_document_matrix)

```

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!

```

	0	1	2	3	4	5	6	7
algorithmic	0	0	1	0	0	0	1	1
approach	0	0	0	0	0	0	1	0
derivative	1	0	0	0	0	1	0	0
efficiency	0	0	1	0	0	0	0	0
financial	0	0	0	1	0	0	0	0
high-frequency	0	0	0	0	1	0	1	0
impact	0	0	1	0	0	0	1	0
integrating	0	0	0	0	0	1	0	0
liquidity	0	0	0	0	1	0	0	0
management	1	0	0	0	0	0	0	0
market	0	0	1	0	0	0	1	0
method	0	0	0	1	0	0	0	1
modeling	0	0	0	1	0	0	0	0
optimization	0	1	0	0	0	1	0	1
portfolio	0	1	0	0	0	1	0	1
provision	0	0	0	0	1	0	0	0

quantitative	0	0	0	1	0	0	0	1
risk	1	0	0	0	0	0	0	0
strategy	0	1	0	0	0	1	1	0
technique	0	1	0	0	0	0	0	0
trading	1	0	1	0	1	0	1	1

This matrix shows the relationships between the terms (rows) and documents (columns). For example:

- The term “algorithmic” appears (value of 1) in documents 3, 7 and 8.
- The term “trading” appears in documents 1, 3, 5, 7 and 8.
- Document 1 contains the terms “derivatives”, “management”, “risk” and “trading”.

This matrix is the foundation for applying LSA or other dimensionality reduction techniques.

1.2.1 2.1 How LSA relate to SVD

LSA uses Singular Value Decomposition (SVD) at its core to find the hidden relationships between terms and concepts. Singular Value Decomposition is a fundamental concept in linear algebra and a powerful technique used in many areas, including machine learning and natural language processing. Here’s how it works:

1. Document-Term Matrix: LSA starts with a document-term matrix, where each row represents a document and each column represents a term (word). The cells contain the frequency of each term in each document.
2. SVD Application or Matrix Factorization: SVD is applied to Document-Term matrix. SVD is a way to factorize (break down) a matrix into three separate matrices. It decomposes the matrix into three matrices:
 - U : Represents document-to-concept similarity.
 - Σ : A diagonal matrix containing singular values, representing the importance of each concept.
 - V : Represents term-to-concept similarity.
 - The Formula: The relationship between the original matrix (A) and the decomposed matrices is: $A = U\Sigma V^T$ (where V^T is the transpose of V).
3. Dimensionality Reduction: By keeping only the largest singular values (and corresponding columns/rows in U and V), we reduce the number of dimensions (concepts) while preserving the most important information. This is similar to Principal Component Analysis (PCA).
4. Concept Space: The resulting matrices represent documents and terms in a new “concept space”. Words with similar meanings will be closer together in this space, even if they don’t appear in the same documents.

In essence, SVD in LSA helps to:

- **Uncover latent relationships:** Identify underlying concepts that are not explicitly present in the text.
- **Reduce noise:** Filter out less important information and focus on the core meaning.
- **Improve information retrieval:** Represent documents and queries in a way that captures semantic similarity.

This process allows LSA to go beyond simple keyword matching and understand the meaning of text at a deeper level.

Let's now proceed with python code where we will use `TruncatedSVD` from `scikit-learn`, essentially performing Singular Value Decomposition (SVD) under the hood. `TruncatedSVD` is an efficient implementation of SVD specifically designed for dimensionality reduction. It calculates the SVD and then keeps only the top `n_components` singular values and their corresponding vectors. This truncated version of the SVD is what produces the `lsa_matrix`, which represents our data in a lower-dimensional concept space.

Important: The output of LSA can be different each time we run the code. This is due to the random initialization of the `TruncatedSVD` algorithm. The variation is related to the nature of Singular Value Decomposition (SVD) and how LSA uses it. Here's a breakdown of the key factors:

- **Rotation in Concept Space:** SVD essentially finds the best way to represent our data in a new "concept space." The components (or dimensions) in this space are determined by finding the directions of greatest variance in the original data. However, there is often some flexibility in how these components are oriented (rotated). Think of it like rotating a 3D object – the object remains the same, but its projection onto the axes changes.
- **Sign Ambiguity:** The signs (positive or negative) of the values in the LSA output can flip without changing the underlying meaning. This is because the direction of a component is arbitrary. Imagine a component representing "financial modeling" – it doesn't matter if it points in one direction or the opposite direction; it still captures the same concept.
- **Sensitivity to Small Changes:** SVD can be sensitive to small changes in the input data. Even minor variations in word frequencies or the addition/removal of a few documents can lead to some degree of rotation in the concept space, resulting in different term loadings.
- **Random Initialization:** As mentioned before, the `TruncatedSVD` algorithm often uses random initialization, which can contribute to variations in the output.

While these variations might seem concerning, they usually don't drastically change the overall interpretation of the LSA results. The relative positions of terms in the concept space and the general patterns of similarity are generally preserved. To ensure consistent results, we will set the `random_state` parameter to a fixed value when creating the `TruncatedSVD` object. This will produce the same output each time we run the code.

```
[8]: # Get the terms (words) from the term-document matrix
terms = term_document_matrix.index

# Create an LSA model and fit to term_document_matrix
lsa = TruncatedSVD(n_components=2, random_state=42)
lsa_matrix = lsa.fit_transform(term_document_matrix)

# Create DataFrame for LSA results with term names and print it
df_lsa = pd.DataFrame(lsa_matrix, index=terms)
print(df_lsa)
```

	0	1
algorithmic	1.487743	-0.457823
approach	0.578476	-0.378199

derivative	0.492907	0.525547
efficiency	0.427052	-0.387020
financial	0.099332	0.155108
high-frequency	0.812589	-0.596790
impact	1.005528	-0.765219
integrating	0.286096	0.555942
liquidity	0.234112	-0.218591
management	0.206810	-0.030394
market	1.005528	-0.765219
method	0.581547	0.462505
modeling	0.099332	0.155108
optimization	1.015624	1.343809
portfolio	1.015624	1.343809
provision	0.234112	-0.218591
quantitative	0.581547	0.462505
risk	0.206810	-0.030394
strategy	1.111885	0.658213
technique	0.247313	0.480471
trading	1.928666	-0.706808

Interpreting the results of LSA involves understanding the new semantic space created by the dimensionality reduction and analyzing the relationships between terms and documents in this space. Here is how to interpret the results: - Interpreting the Columns: These are also known as Components or Concepts or Dimensions. Think of each component as representing a hidden “concept” or “topic” within documents. - Interpreting the rows: Each row in this matrix represents a term from our original matrix, but now projected onto a 2-dimensional concept space (because we chose `n_components=2`). - Interpreting the values: - The two values in each row represent the coordinates of that term in the new concept space. Terms with similar coordinates are closer together in the concept space, indicating a semantic relationship. In our example first row represents the term “algorithmic”. Its coordinates are approximately (1.488 -0.458). If another term, has similar coordinates, it suggests that these terms are related in the context of our documents. - The values represent the loading of that term on the corresponding component. The loadings tell us how much each term contributes to that concept. For example, “algorithmic” and “trading” have high loadings on Component 1, suggesting that Component 1 represents the concept of “algorithmic trading”. A higher value (positive or negative) indicates a stronger association between the term and that component.

From the output of our code we can interpret the relationships between terms by looking at their coordinates and draw some preliminary observations:

- **Distinct Clusters:**

- Trading Cluster: “algorithmic”, “high-frequency”, “impact”, “market”, and “trading” have high values for component 0 and mostly negative values for component 1. This reinforces the idea that component 0 represents concepts related to automated and high-frequency trading, and their potential impact on the market.
- Portfolio and Optimization Cluster: “optimization” and “portfolio” form a clear cluster with high positive values for both components. This suggests a strong association between these terms, likely related to portfolio optimization techniques.
- Derivatives and Method-Quantitative Cluster: Grouping “method”, “quantitative”, and

“derivative” together suggests a potential connection between quantitative methods and their application to derivatives.

- Management-Risk-Liquidity-Provision Cluster: Combining “management”, “risk”, “liquidity”, and “provision” suggests a cluster related to managing financial risk and ensuring liquidity. This also seems plausible, as these concepts are often intertwined in financial contexts, particularly when dealing with derivatives or trading activities.

- **Other Observations:**

- “Trading” as a central concept: “trading” has the highest value for component 0, indicating its importance and potential connection to various forms of trading discussed in the documents.
- Distinct Pairs: The pair of terms “financial” and “modeling” form very distinct pair due to their identical coordinates. Further analysis is needed to determine their relationship to other terms and clusters.
- Overlap between clusters: Some terms, like “strategy” and “technique”, have moderate values in multiple components, suggesting potential overlap or connections between different clusters.

This quick analysis gives somewhat good overview of the relationships between terms based on the LSA results.

We can now plot the coordinates to visualise them and confirm if these proposed clusters appear close together in visual representation. To achieve this, we will use `plotly.graph_objects` module imported as `go`:

```
[9]: # Create a list of hover texts with labels and coordinates
hover_texts = []
for row in df_lsa.values:
    terms = df_lsa.index[(df_lsa[0] == row[0]) & (df_lsa[1] == row[1])]
    terms_str = ', '.join([f'{term}' for term in terms])
    coord = f"({row[0]:.5f}, {row[1]:.5f})"
    hover_texts.append(f"Terms: {terms_str}<br>{coord}")

# Create the scatter plot
fig = go.Figure(go.Scatter(
    x=df_lsa[0],
    y=df_lsa[1],
    mode='markers',
    marker=go.scatter.Marker(
        size=10,
        color='blue',
        opacity=0.8
    ),
    hovertemplate='%{text}<extra></extra>',
    text=hover_texts
))

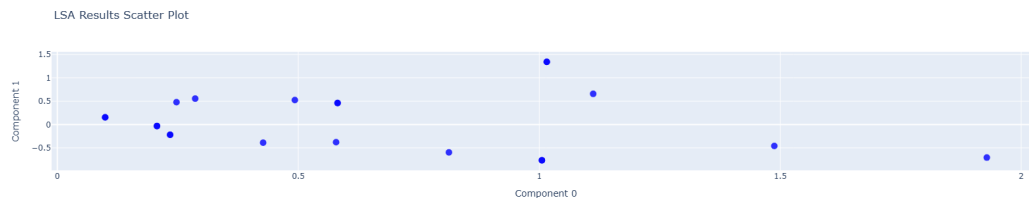
# Set axis labels and title
fig.update_layout(
```

```

axis_title="Component 0",
axis_title="Component 1",
title="LSA Results Scatter Plot"
)

fig.show()

```



Plotly plots are interactive by default. We can use its interactive features to pan/zoom and hover/select to see tooltips with term labels and coordinates.

Now looking at this plot it seems that clusters are not that easy to visualise. It seems that we were too focused on the moderate values of term coordinates, without paying enough attention to the precise distances between them. To determine clusters more accurately, we should use a more formal methods. By using these methods, we can avoid subjective interpretations and define clusters more precisely based on quantitative measures of similarity or distance. We will consider these methods later in this lesson.

1.2.2 2.2 Important considerations with LSA and SVD

Choosing the optimal number of components for LSA Choosing the optimal number of components (`n_components`) for LSA is a crucial step. Here are some considerations:

- **Start small:** Begin with a small number of components (e.g., 2, 3, or 5) to see if it gives a good initial understanding of the relationships between terms and documents. This can help to get a feel for the data and identify potential clusters or topics.
- **Iteratively increase:** Gradually increase the number of components and observe how the interpretation of the results changes. Look for a point where adding more components doesn't significantly improve the interpretability or reveal new meaningful relationships.
- **Consider the size of corpus:** For smaller corpora, fewer components might suffice to avoid overfitting. For larger corpora, more components could capture more subtle semantic relationships.
- **Focus on semantic coherence:** Evaluate the semantic coherence of the resulting topics or clusters. Do the terms within each cluster make sense together? Are the clusters distinct and meaningful? If the coherence decreases as we add more components, it might be a sign that we are starting to overfit.
- **Balance dimensionality reduction with information loss:** LSA involves a trade-off between reducing dimensionality and preserving information. We should try to find a sweet spot where we have a manageable number of components while still capturing the most important semantic relationships.

In summary, there is no one-size-fits-all answer for the optimal number of components in LSA. It depends on specific data and goals. We would need to experiment with different values, carefully evaluate the results, and use domain knowledge to guide our choice.

Understanding limitations of LSA and SVD As we observe from above example some terms are clustered as they appear in particular document. For example “algorithmic”, “high-frequency”, “impact”, “market”, and “trading” are similar terms making one cluster. And they are all from document 7.

This is crucial point about Latent Semantic Analysis (LSA) and its potential limitations. If LSA consistently clusters terms solely based on their co-occurrence within the same document, it raises questions about its effectiveness in capturing broader semantic relationships.

Here’s a breakdown of why this might happen and what it means for LSA:

- **Document-Specific Context:** LSA primarily relies on word co-occurrence patterns within a corpus. If terms frequently appear together in a single document but not across the entire corpus, LSA might interpret them as semantically related even if their broader contextual meanings differ significantly.
- **Limited Corpus Size or Diversity:** A small or homogenous corpus can exacerbate this issue. If documents within the corpus lack diversity in topics and language use, LSA might struggle to identify broader semantic relationships beyond document-level co-occurrences.
- **Data Sparsity:** In LSA, the term-document matrix can be very sparse, especially with a large vocabulary. This sparsity can sometimes lead to the model focusing on local co-occurrence patterns within documents rather than global semantic relationships.

So, what’s the point of LSA if it primarily identifies document-level clusters? While the scenario we encounter isn’t ideal, LSA can still offer valuable insights:

- **Document Summarization:** LSA can be effective in identifying key terms and concepts within individual documents, which is useful for summarization and topic extraction.
- **Dimensionality Reduction:** Even if clusters are document-specific, LSA effectively reduces the dimensionality of the data, making it easier to analyze and visualize.
- **Basis for Other Techniques:** LSA can serve as a foundation for more advanced topic modeling techniques that address its limitations, such as Latent Dirichlet Allocation (LDA).

To improve LSA’s performance and capture broader semantic relationships:

- **Increase Corpus Size and Diversity:** Include a wider range of documents on different topics with diverse language use.
- **Preprocessing and Feature Engineering:** Carefully preprocess the text (e.g., removing stop words, stemming/lemmatization) and consider using different weighting schemes (e.g., Term Frequency-Inverse Document Frequency or TF-IDF in short, a numerical statistic used in information retrieval and text mining to reflect how important a word is to a document in a collection or corpus.) to emphasize more informative terms.
- **Explore Other Techniques:** Consider using alternative topic modeling techniques like LDA, which explicitly models the distribution of topics within documents.

In summary, while LSA can be useful for document-level analysis, its limitations in capturing broader semantic relationships should be considered. Ensuring a diverse and representative corpus and exploring alternative techniques can help overcome these limitations and unlock the full

potential of topic modeling.

1.3 3 Compute Similarity Measures

In this subsection we will apply several of the similarity measures to our LSA results to further analyse terms in LSA space.

It is generally not best practice to rely solely on one similarity measure, especially when exploring complex datasets or tasks. Different similarity measures have different strengths and weaknesses and may capture different aspects of similarity.

1.3.1 3.1 Cosine similarity measure between terms

Cosine similarity is a common way to measure the similarity between vectors in the LSA space. The following code helps to quantify the semantic similarity between terms within the first distinct cluster (terms: “algorithmic”, “high-frequency”, “impact”, “market”, “trading”) based on their representations in the LSA concept space:

```
[10]: # Extract term vectors for first distinct cluster
terms_cl1 = ["algorithmic", "high-frequency", "impact", "market", "trading"]
term_vectors_cl1 = df_lsa.loc[terms_cl1]

# Calculate cosine similarity
similarity_matrix = cosine_similarity(term_vectors_cl1)

# Convert to DataFrame for better visualization
similarity_df = pd.DataFrame(similarity_matrix, index=terms_cl1,
                              columns=terms_cl1)

# Print the results for the first distinct cluster with labels
print("Cosine similarity for first distinct cluster:")
print(similarity_df)
```

Cosine similarity for first distinct cluster:

	algorithmic	high-frequency	impact	market	trading
algorithmic	1.000000	0.944434	0.938692	0.938692	0.998609
high-frequency	0.944434	1.000000	0.999855	0.999855	0.960449
impact	0.938692	0.999855	1.000000	1.000000	0.955562
market	0.938692	0.999855	1.000000	1.000000	0.955562
trading	0.998609	0.960449	0.955562	0.955562	1.000000

This matrix shows the pairwise cosine similarity between the terms in the first distinct cluster. The values range from 0 to 1, where 1 indicates perfect similarity and 0 indicates no similarity. Here are some observations:

- High similarity within the cluster: Most of the values in the matrix are close to 1, indicating that the terms in this cluster are highly semantically similar. This is expected since they were identified as a cluster in the LSA analysis.
- “Impact” and “market” are perfectly similar: The cosine similarity between “impact” and “market” is 1, suggesting that they are considered synonymous in the context of this corpus.

- “Algorithmic” and “trading” are very similar: The cosine similarity between “algorithmic” and “trading” is very high (0.998), indicating a strong semantic connection between these terms. This makes sense as “algorithmic trading” is a common concept.
- “High-frequency” is slightly less similar: “High-frequency” has slightly lower similarity scores compared to other terms, suggesting that it might have a slightly different meaning or context within this cluster.

This analysis confirms that the terms in the first distinct cluster are indeed semantically related based on their cosine similarity.

Let’s now compute cosine similarity for terms in the fourth cluster:

```
[11]: # Extract term vectors for fourth cluster
terms_cl4 = ["management", "risk", "liquidity", "provision"]
term_vectors_cl4 = df_lsa.loc[terms_cl4]

# Calculate cosine similarity
similarity_matrix = cosine_similarity(term_vectors_cl4)

# Convert to DataFrame for better visualization
similarity_df = pd.DataFrame(similarity_matrix, index=terms_cl4,
    ↪ columns=terms_cl4)

# Print the results for the fourth cluster with labels
print("Cosine similarity for fourth cluster:")
print(similarity_df)
```

Cosine similarity for fourth cluster:

	management	risk	liquidity	provision
management	1.000000	1.000000	0.822387	0.822387
risk	1.000000	1.000000	0.822387	0.822387
liquidity	0.822387	0.822387	1.000000	1.000000
provision	0.822387	0.822387	1.000000	1.000000

Here we have the following:

- Perfect similarity: “Management” and “risk” have a cosine similarity of 1, indicating they are treated as highly related or even synonymous within this context. Similarly, “liquidity” and “provision” also show perfect similarity. This is not surprising because cosine similarity measures the angle between two vectors. Since “Management” and “risk” have identical coordinates, they point in the same direction, resulting in 0-degree angle and cosine similarity of 1. Similarly, “liquidity” and “provision” also form another pair of identical vectors.
- Strong but not perfect similarity: While the other terms exhibit strong similarity, it’s not perfect (cosine similarity of 0.82). This suggests a degree of semantic overlap but also some distinction in their meanings within the documents.

This cluster clearly indicates a focus on managing risk, potentially highlighting the importance of liquidity provision in mitigating financial risks.

We can continue this approach of selecting specific clusters and analyzing cosine similarity within

those groups. This allows us to focus on terms that are already known to be related and uncover finer-grained semantic connections.

By using a more targeted approach, we can gain more meaningful insights from cosine similarity analysis and better understand the semantic structure of our data.

One way to explore relationship between clusters is to calculate the centroid of each cluster and then measure the cosine similarity between the centroids. This can help us understand how different topics or concepts relate to each other.

Earlier we identified four distinct clusters and one pair Distinct Pair: The pair of terms “financial” and “modeling”. Let’s further analyse this pair to determine their relationship to other terms and clusters.

```
[12]: # Extract term vectors for distinct pair
terms_pair = ["financial", "modeling"]
term_vectors_pair = df_lsa.loc[terms_pair]

# Extract term vectors for second and third cluster
term_vectors_cl2 = df_lsa.loc[["optimization", "portfolio"]]
term_vectors_cl3 = df_lsa.loc[["derivative", "method", "quantitative"]]

# Calculate centroid for the distinct pair
centroid_pair = term_vectors_pair.mean()

# Store cluster term vectors in a list and calculate centroids for each cluster
cluster_term_vectors = [term_vectors_cl1, term_vectors_cl2, term_vectors_cl3,
    ↪ term_vectors_cl4]
centroids = [term_vectors.mean() for term_vectors in cluster_term_vectors]

# Calculate cosine similarity between the distinct pair and each cluster
    ↪ centroid
similarities = [cosine_similarity(centroid_pair.values.reshape(1, -1), centroid.
    ↪ values.reshape(1, -1)) for centroid in centroids]

# Print the similarities
for i, similarity in enumerate(similarities):
    print(f"Cosine similarity between distinct pair and cluster {i+1}:",
    ↪ similarity)
```

```
Cosine similarity between distinct pair and cluster 1: [[0.08406783]]
```

```
Cosine similarity between distinct pair and cluster 2: [[0.9969915]]
```

```
Cosine similarity between distinct pair and cluster 3: [[0.96054718]]
```

```
Cosine similarity between distinct pair and cluster 4: [[0.05551994]]
```

These results show the cosine similarity between the distinct pair and each of the four clusters. Here’s how to interpret them:

- Strongest similarity with cluster 2: The distinct pair has the highest cosine similarity (0.997) with cluster 2. This suggests that the pair is very closely related to the concepts or terms represented by cluster 2.
- Moderate similarity with cluster 3: There's a moderate similarity (0.961) with cluster 3, indicating some overlap in meaning or context.
- Weak similarity with clusters 1 and 4: The pair shows very weak similarity with clusters 1 (0.084) and 4 (0.056), suggesting that these clusters represent distinct concepts that are not closely related to the pair.

Conclusion: Based on these results, we can conclude that the distinct pair is most strongly associated with cluster 2, with some moderate connection to cluster 3. It's likely that the pair shares a significant semantic overlap with the terms in cluster 2.

1.3.2 3.2 Cosine similarity between documents

Measuring cosine similarity between entire documents is one of the common applications of cosine similarity in natural language processing and information retrieval.

To do this we first need to get **Document vectors**. There are few techniques to do this such as getting LSA vectors, Word embeddings and Term Frequency-Inverse Document Frequency (TF-IDF).

In our lesson we will focus on TF-IDF vectors as these capture the importance of words in a document relative to a collection of documents. The following code calculates the cosine similarity between the documents using TF-IDF and presents the output in a dataframe.

```
[13]: # Create a TfidfVectorizer object and fit to documents
tfidf_vectorizer = TfidfVectorizer(tokenizer=preprocess_text,
    token_pattern=None)
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)

# calculate the cosine similarity matrix
cosine_sim = cosine_similarity(tfidf_matrix)

# Present TF-IDF matrix in DataFrame form
short_labels = ['Doc ' + str(i+1) for i in range(len(documents))]
df = pd.DataFrame(cosine_sim, index=short_labels, columns=short_labels)
df
```

```
[13]:
```

	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5	Doc 6	Doc 7	
Doc 1	1.000000	0.000000	0.100709	0.000000	0.104403	0.223556	0.085795	\
Doc 2	0.000000	1.000000	0.000000	0.000000	0.000000	0.541226	0.154369	
Doc 3	0.100709	0.000000	1.000000	0.000000	0.100709	0.000000	0.589201	
Doc 4	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	
Doc 5	0.104403	0.000000	0.100709	0.000000	1.000000	0.000000	0.277083	
Doc 6	0.223556	0.541226	0.000000	0.000000	0.000000	1.000000	0.136797	
Doc 7	0.085795	0.154369	0.589201	0.000000	0.277083	0.136797	1.000000	
Doc 8	0.100002	0.359862	0.256616	0.419796	0.100002	0.318899	0.218614	

	Doc 8
Doc 1	0.100002
Doc 2	0.359862
Doc 3	0.256616
Doc 4	0.419796
Doc 5	0.100002
Doc 6	0.318899
Doc 7	0.218614
Doc 8	1.000000

Here `TfidfVectorizer` is initialized with the same `preprocess_text` function we used previously for preprocessing document texts during LSA analysis. This function is passed as an argument to the `tokenizer` parameter inside `TfidfVectorizer`. By using the `preprocess_text` function as the tokenizer, we ensure that the TF-IDF calculation is based on the lemmatized and cleaned tokens, which can lead to more meaningful and accurate results. The `TfidfVectorizer` will apply this function to each document in the corpus to convert the raw text into a sequence of tokens (words or terms).

Both `TfidfVectorizer` and `CountVectorizer` that we used earlier are used in `scikit-learn` for text analysis to convert a collection of text documents into a matrix of numerical features. However, they differ in how they represent the importance of words in the documents:

- **CountVectorizer:** Creates a matrix where each entry represents the number of times a word appears in a document (term frequency). It's a simple way to represent text data numerically, but it doesn't consider the relative importance of words across the entire corpus. `CountVectorizer` focuses on the local importance of words within a document.
- **TfidfVectorizer:** Creates a matrix where each entry represents the TF-IDF score of a word in a document. TF-IDF (Term Frequency-Inverse Document Frequency) takes into account both the frequency of a word in a document and its rarity across the entire corpus. This means that words that are frequent in a document but rare in the corpus will have higher TF-IDF scores, indicating their importance for that specific document. `TfidfVectorizer` focuses on the global importance of words relative to the entire corpus.

Let's now analyse the code output. The above DataFrame shows the cosine similarity between the documents. Here are some observations based on the cosine similarity scores:

- Document 6 and Document 2 are the most similar (0.54). This makes sense as both documents discuss portfolio optimization.
- Document 7 and Document 3 are also quite similar (0.59), which is expected as they both cover algorithmic trading.
- Document 8 seems to be moderately similar to several documents, including Document 2 (0.36), Document 6 (0.32), and Document 4 (0.42). This is because it covers a broader range of topics, including algorithmic trading and portfolio optimization.
- Document 1 and Document 5 have a low similarity (0.10) with most other documents as they discuss risk management and liquidity provision, respectively, which are relatively broad topics.

Here are some potential insights we could draw:

- Documents related to portfolio optimization (2 and 6) form a cluster of related topics.

- Algorithmic trading (3 and 7) is another cluster of related concepts.
- Document 8 acts as a bridge between these two clusters, indicating some overlap between the topics.
- Risk management (1) and liquidity provision (5) are relatively independent concepts within this dataset.

Please note that these are just initial observations. One can delve deeper into the documents themselves to understand the nuances of their similarity and how the specific terms contribute to the cosine similarity scores.

Remember that cosine similarity is just one measure of similarity. Depending on specific needs and the nature of data, we might need to explore other similarity measures as well.

1.3.3 3.3 Jaccard index

Let's now consider using Jaccard index as measure to estimate similarity between our documents. In the following code snippet we constructed `jaccard_similarity` function that calculates the Jaccard similarity between two documents which are pre-processed. Inside this function we apply `intersection` and `union` built-in methods available for set objects in Python. And then Jaccard index is calculated by simply dividing size of the intersection by the size of union: `intersection / union`. We will apply this function to all possible pairs in our document corpus to compute all pairwise indexes:

```
[14]: # Sample documents
documents = [
    "Risk management in derivatives trading",
    "Portfolio optimization techniques and strategies",
    "Algorithmic trading and its impact on market efficiency",
    "Quantitative methods for financial modeling",
    "High-frequency trading and liquidity provision",
    "Integrating Derivatives into Portfolio Optimization Strategies",
    "High-Frequency Trading Strategies: Algorithmic Approaches and Market_
↪Impact",
    "Quantitative Methods for Algorithmic Trading and Portfolio Optimization"
]

# Download stopwords, WordNet resources and create a Lemmatizer Object
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')
lemmatizer = WordNetLemmatizer()

# Preprocessing function
def preprocess_text(text):
    text = re.sub(r'[\w\s-]', '', text) # Remove punctuation except hyphens
    tokens = nltk.word_tokenize(text.lower())
    tokens = [lemmatizer.lemmatize(token) for token in tokens] # Lemmatize
    stop_words = set(stopwords.words('english')) # define stop_words as a set_
↪of english stop words
```

```

    tokens = [token for token in tokens if token not in stop_words] # Remove
    ↪ stop words
    return tokens

# Function to calculate Jaccard similarity between two documents
def jaccard_similarity(doc1, doc2):
    # Preprocess documents
    tokens1 = set(preprocess_text(doc1))
    tokens2 = set(preprocess_text(doc2))

    # Calculate Jaccard similarity
    intersection = len(tokens1.intersection(tokens2))
    union = len(tokens1.union(tokens2))
    return intersection / union

# Calculate Jaccard similarity matrix
jaccard_sim = [[jaccard_similarity(doc1, doc2) for doc2 in documents] for doc1
    ↪ in documents]

# Present Jaccard similarity matrix in DataFrame form
short_labels = ['Doc ' + str(i+1) for i in range(len(documents))]
df = pd.DataFrame(jaccard_sim, index=short_labels, columns=short_labels)
df

```

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!

```

```

[14]:
      Doc 1  Doc 2  Doc 3  Doc 4  Doc 5  Doc 6  Doc 7
Doc 1  1.000000  0.00  0.125000  0.00  0.142857  0.125000  0.100000 \
Doc 2  0.000000  1.00  0.000000  0.00  0.000000  0.500000  0.100000
Doc 3  0.125000  0.00  1.000000  0.00  0.125000  0.000000  0.500000
Doc 4  0.000000  0.00  0.000000  1.00  0.000000  0.000000  0.000000
Doc 5  0.142857  0.00  0.125000  0.00  1.000000  0.000000  0.222222
Doc 6  0.125000  0.50  0.000000  0.00  0.000000  1.000000  0.090909
Doc 7  0.100000  0.10  0.500000  0.00  0.222222  0.090909  1.000000
Doc 8  0.111111  0.25  0.222222  0.25  0.111111  0.222222  0.181818

      Doc 8
Doc 1  0.111111
Doc 2  0.250000
Doc 3  0.222222
Doc 4  0.250000

```

```
Doc 5  0.111111
Doc 6  0.222222
Doc 7  0.181818
Doc 8  1.000000
```

Here we observe: - Strong similarity between Documents 2 and 6 with highest similarity score (0.50) and also between documents 3 and 7 with also similarity score (0.50). - Moderate similarity: Document 8 again shows moderate similarity with Documents 2 (0.25), 4 (0.25) and 6 (0.22). - Low similarity: Documents 1 and 5 again exhibit low similarity.

Comparison with Cosine similarity: The general patterns of similarity are relatively consistent with the cosine similarity results. However, Jaccard index tends to produce lower similarity scores overall, as it focuses on the overlap of terms and does not consider term frequency or inverse document frequency.

1.4 Conclusion

By the end of this lesson, you should have a solid understanding of how to use Python and Google Trends to analyze search trends. You should be able to fetch and compare the interest over time for multiple search terms, analyze interest by region, discover top and rising related queries. In this lesson we also explored how to leverage data science and machine learning for financial analysis, with a focus on analyzing search trends from Google Trends data. It covered techniques like TF-IDF and LSA to extract insights from text, and demonstrated how to gather and analyze Google Trends data using Python and the `pytrends` library. The notebook also discussed the application of these techniques for sentiment analysis, risk management, and portfolio optimization.

References

- Vrba, G. (2018). Timing The Market With Google Trends Search Volume Data. [online] Seeking Alpha. Available at: <https://seekingalpha.com/article/4202781-timing-market-google-trends-search-volume-data>.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Taborda, B., Maria de Almeida, A., Carlos Dias, J., Batista, F., & Ribeiro, R. (2023). SA-MAIS: Hybrid automatic sentiment analyser for stock market. *Journal of Information Science*, 0(0).

<https://doi.org/10.1177/01655515231171361>

- **Required pages:** The entire content
- **Estimated reading time:** 25 minutes

2. Liu J-X, Leu J-S, Holst S. 2023. Stock price movement prediction based on Stocktwits investor sentiment using FinBERT and ensemble SVM. *PeerJ Comput. Sci.* 9:e1403 <http://doi.org/10.7717/peerj-cs.1403>

- **Required pages:** The entire content
- **Estimated reading time:** 40 minutes

Applications of Alternative Data

In this lesson we consider applications of alternative data. Alternative Data in finance refers to non-traditional data sources used to gain insights into market trends, asset pricing, and investment decisions. This contrasts with traditional data like financial statements, company filings, and market prices. Text data from social media falls under this category as it reflects public sentiment, market chatter, and news dissemination that can influence asset prices. Specifically, Twitter and StockTwits data are considered valuable sources of alternative data in financial engineering due to their real-time nature and focus on financial discussions and stock market activity.

```
In [1]: # Load Libraries
import datetime
import matplotlib.pyplot as plt
import os
import pandas as pd
import plotly.express as px
import re
import requests
import zipfile

from datetime import datetime, timedelta
```

1 What is Twitter Data

Twitter data refers to the vast amount of information generated and shared on the Twitter platform. Twitter data is a vast collection of information generated by users on the platform, including tweets, profiles, messages, trends, media, and interactions. It's primarily available in JSON format through the Twitter API, though other methods like web scraping and third-party tools can also be used. Access to historical data is often limited, and an Academic Research Track offers greater access for researchers. Twitter data is a rich resource for understanding and analyzing the online world. This data is valuable for social media analytics, market research, business intelligence, news, public health, political science, and academic research, allowing insights into user behavior, public opinion, and real-time events.

Data Structure and Format:

- JSON: The Twitter API primarily provides data in JSON (JavaScript Object Notation) format. JSON is a lightweight data-interchange format that is easy for humans to read and write and for machines to parse and generate. Each tweet, user profile, or other data object is represented as a JSON object with various attributes and values.

- CSV/TSV: Twitter data can also be exported or converted into CSV (Comma-Separated Values) or TSV (Tab-Separated Values) formats for analysis in spreadsheet software or other tools.
- Databases: For large-scale analysis and storage, Twitter data is often loaded into databases like MySQL, PostgreSQL, or NoSQL databases like MongoDB.

Twitter Data Access and Limitations:

- Twitter API: While the Twitter API provides a powerful way to access data, there are limitations and restrictions. These include rate limits on the number of requests you can make, access to only a subset of historical data (typically the past 7 days for the standard API), and requirements for developer accounts and API keys.
- Academic Research Track: For academic researchers, Twitter offers an "Academic Research Track" with elevated access levels and greater data availability. This track requires a detailed research proposal and approval process.
- Data Bias and Ethical Considerations: It's important to be aware of potential biases in Twitter data, such as the overrepresentation of certain demographics or viewpoints. Ethical considerations are crucial, including respecting user privacy, obtaining informed consent when necessary, and avoiding the spread of misinformation. Twitter data is inherently biased towards active users and publicly shared opinions. This dataset may not represent the views of the entire investor population.
- Data Sparsity and Quality: The dataset may contain gaps or periods where tweet volume is low, which can affect the analysis. It's essential to verify the data quality and address any potential issues like missing values or inconsistencies before conducting analysis.

There are several options for downloading ready-made Twitter datasets, several avenues exist for obtaining ready-made Twitter data, catering to both researchers and general users, with varying levels of access and data specifics:

- For researchers: The Twitter Academic Research Track offers the most comprehensive access, but requires application and approval.
- Public options: Platforms like Kaggle, figshare, Zenodo, Data.World, Hugging Face, IEEE DataPort and Mendeley Data host a variety of datasets on different topics, readily available for download. Kaggle is the most prominent community-run platform among the ones mentioned. IEEE DataPort is more focused on research data sharing within specific fields. Hugging Face is a community-driven platform specializing in NLP and related areas.
- Specialized sources: Government agencies and organizations may offer datasets relevant to specific events or topics.

When using these platforms, perhaps most important is to adhere to responsible data practices and prioritize legality, user protection, and ethical conduct, ensuring trust and positive societal impact. Data licensing, user privacy, and ethical considerations are essential

for responsible data use. Licensing ensures legal compliance, privacy safeguards personal information, and ethics guide fair and respectful data practices. These aspects are crucial for publicly available data as well, requiring adherence to license terms, anonymization for privacy, and awareness of potential biases for ethical analysis.

1.1 Getting Twitter Data

In this lesson we explore Stock Market Tweets Data from IEEE Dataport available at <https://dx.doi.org/10.21227/g8vy-5w61>. IEEE DataPort is a curated repository for research data, primarily focusing on engineering and technology-related fields. It is operated by the Institute of Electrical and Electronics Engineers (IEEE) and encourages researchers to share their datasets to advance knowledge in their respective domains. This dataset contains a collection of tweets related to the stock market, specifically focusing on the S&P 500 index. It aims to provide data for researchers and practitioners interested in analyzing the relationship between social media sentiment and stock market movements.

This dataset is available under the Creative Commons Attribution 4.0 International (CC BY 4.0) license. This license allows for the free use, sharing, adaptation, and redistribution of the data, as long as appropriate credit is given to the original creators.

The dataset covers tweets from April 9th, 2020 to July 16th, 2020. This time range provides a substantial amount of data for analysis and covers various market conditions. The data was collected using the S&P 500 tag (#SPX500), the references to the top 25 companies in the S&P 500 index, and the Bloomberg tag (#stocks). The data is packaged in ZIP archive in two CSV files: 'tweets_labelled_09042020_16072020.csv' consists of 5,000 tweets selected using random sampling. Out of those 5,000 tweets, 1,300 were manually annotated in positive, neutral, or negative classes. The second file 'tweets_remaining_09042020_16072020.csv' contains remainder of tweets.

The following code snippet is designed to extract a ZIP file containing Twitter data and then locate all the CSV files within the extracted folder for further processing. The code iterates through the list of CSV files containing Twitter data, reads each file into a pandas DataFrame, standardizes the column name for tweet text, and then combines all the DataFrames into a single DataFrame called `twitter_df`:

```
In [2]: # Extract the ZIP file in folder named 'extracted_tweets'
with zipfile.ZipFile('tweets.zip', 'r') as zip_ref:
    zip_ref.extractall('extracted_tweets')

# Get a List of CSV files in the inner 'tweets' folder
tweets_folder = 'extracted_tweets/tweets'
csv_files = [f for f in os.listdir(tweets_folder) if f.endswith('.csv')]

# Loop through CSV files and read them into DataFrames
dfs = []
for csv_file in csv_files:
```

```

file_path = os.path.join(tweets_folder, csv_file)
df = pd.read_csv(file_path, sep=';')

# Rename 'full_text' or 'text' column to a 'tweet_text' common name
if 'full_text' in df.columns:
    df = df.rename(columns={'full_text': 'tweet_text'})
elif 'text' in df.columns:
    df = df.rename(columns={'text': 'tweet_text'})

dfs.append(df)

# Concatenate all DataFrames into a single DataFrame and display it
twitter_df = pd.concat(dfs, ignore_index=True)
twitter_df

```

Out[2]:

	id	created_at	tweet_text	sentiment
0	77522	2020-04-15 01:03:46+00:00	RT @RobertBeadles: Yo 🌟 WIN 1,000 Mon...	positive
1	661634	2020-06-25 06:20:06+00:00	#SriLanka surcharge on fuel removed! 🇨🇦 🇨🇦 The ...	negative
2	413231	2020-06-04 15:41:45+00:00	Net issuance increases to fund fiscal programs...	positive
3	760262	2020-07-03 19:39:35+00:00	RT @bentboolean: How much of Amazon's traffic ...	positive
4	830153	2020-07-09 14:39:14+00:00	\$AMD Ryzen 4000 desktop CPUs looking 'great' a...	positive
...	...	...	...	...
928668	938668	2020-07-16 00:00:25+00:00	RT @MadMraket: We tend to spend lot of time in...	NaN
928669	938669	2020-07-16 00:00:23+00:00	You \n\n\$SPX \$SPY #ES_F	NaN
928670	938670	2020-07-16 00:00:23+00:00	\$KO Coca-Cola #Options #maxpain Chart, Open In...	NaN
928671	938671	2020-07-16 00:00:06+00:00	Here's a dividends watchlist \nfor the 01/04/0...	NaN
928672	938672	2020-07-16 00:00:00+00:00	\$AAL \$TWTR \$SPCE \$AZN \$ERI \$RCL \$JNJ \$AA \$SNBR...	NaN

928673 rows × 4 columns

1.2 Adjusting time zone in Twitter data

Let's now observe Date and Time formats in 'created_at' column. The "+00:00" in the 'created_at' values indicates a timezone. It represents an offset from Coordinated Universal Time (UTC), which in this case is 0 hours. This means that the timestamps are in UTC itself.

Let's convert this to New York timezone. We do this by using `pd.to_datetime()` function converts the 'created_at' column to pandas DateTime objects. Then we apply chained `.dt.tz_convert('America/New_York')` function which converts the DateTime objects to the New York timezone using the IANA time zone name 'America/New_York'. This ensures that the timestamps are adjusted for daylight saving time transitions in New York:

```
In [3]: # Convert timestamps to the New York timezone
twitter_df['created_at'] = pd.to_datetime(twitter_df['created_at']).dt.tz_convert('America/New_York')
```

	id	created_at	tweet_text	sentiment
0	77522	2020-04-14 21:03:46-04:00	RT @RobertBeadles: Yo 🌟 \nEnter to WIN 1,000 Mon...	positive
1	661634	2020-06-25 02:20:06-04:00	#SriLanka surcharge on fuel removed!\n 📄 📉 \nThe ...	negative
2	413231	2020-06-04 11:41:45-04:00	Net issuance increases to fund fiscal programs...	positive
3	760262	2020-07-03 15:39:35-04:00	RT @bentboolean: How much of Amazon's traffic ...	positive
4	830153	2020-07-09 10:39:14-04:00	\$AMD Ryzen 4000 desktop CPUs looking 'great' a...	positive
...	...	...	...	...
928668	938668	2020-07-15 20:00:25-04:00	RT @MadMraket: We tend to spend lot of time in...	NaN
928669	938669	2020-07-15 20:00:23-04:00	You \n\n\$SPX \$SPY #ES_F	NaN
928670	938670	2020-07-15 20:00:23-04:00	\$KO Coca-Cola #Options #maxpain Chart, Open In...	NaN
928671	938671	2020-07-15 20:00:06-04:00	Here's a dividends watchlist \nfor the 01/04/0...	NaN
928672	938672	2020-07-15 20:00:00-04:00	\$AAL \$TWTR \$SPCE \$AZN \$ERI \$RCL \$JNJ \$AA \$SNBR...	NaN

928673 rows × 4 columns

1.3 Discovering retweets in Twitter data

On Twitter, "RT" is a common abbreviation for "retweet". When a user wants to share someone else's tweet with their own followers, they can retweet it. "RT" is a key indicator of retweets on Twitter. It's used in manual retweets, quote retweets, and is a convention that has become part of Twitter's culture. By searching for "RT" at the beginning of tweets, we can effectively identify retweets in data analysis.

Let's now take our collection of tweets and separate them into two groups: one for retweets and one for original tweets (or other types of tweets that are not retweets). The following code snippet takes `twitter_df` DataFrame containing Twitter data and splits it into two new DataFrames by filtering 'tweet_text' column. Then the code displays both `retweets_df` and `remainder_df` using the `display()` function:

```
In [4]: # Create a DataFrame containing only retweets
retweets_df = twitter_df[twitter_df['tweet_text'].str.startswith('RT')]

# Create a DataFrame containing tweets that are not retweets
remainder_df = twitter_df[~twitter_df['tweet_text'].str.startswith('RT')]

# Display the DataFrames
print("## Retweets DataFrame:")
print("This DataFrame contains only retweets from the Twitter data.")
display(retweets_df)

print("\n## Remainder DataFrame:")
print("This DataFrame contains tweets that are not retweets.")
display(remainder_df)
```

Retweets DataFrame:

This DataFrame contains only retweets from the Twitter data.

	id	created_at	tweet_text	sentiment
0	77522	2020-04-14 21:03:46-04:00	RT @RobertBeadles: Yo 🌟 \nEnter to WIN 1,000 Mon...	positive
3	760262	2020-07-03 15:39:35-04:00	RT @bentboolean: How much of Amazon's traffic ...	positive
5	27027	2020-04-12 17:52:56-04:00	RT @QuantTrend: Reduce your portfolio RISK! GO...	positive
7	392845	2020-06-01 21:12:29-04:00	RT @ArjunKharpal: #Apple has cut the prices of...	negative
8	313771	2020-05-07 00:58:41-04:00	RT @SMA_alpha: The #CDC U.S. New Case data has...	negative
...	...	...	...	...
928659	938659	2020-07-15 20:01:05-04:00	RT @tradewithdough: #India is adding millions ...	NaN
928662	938662	2020-07-15 20:00:57-04:00	RT @WarlusTrades: \$SPX SPY #ES_F #AMD\n\n 🚨 Co...	NaN
928664	938664	2020-07-15 20:00:42-04:00	RT @BerkshireCapGrp: MEDH GAP FILLED. TIME TO ...	NaN
928667	938667	2020-07-15 20:00:26-04:00	RT @CoronaStocks: Boeing is Facing Major Headw...	NaN
928668	938668	2020-07-15 20:00:25-04:00	RT @MadMraket: We tend to spend lot of time in...	NaN

350752 rows × 4 columns

Remainder DataFrame:

This DataFrame contains tweets that are not retweets.

	id	created_at	tweet_text	sentiment
1	661634	2020-06-25 02:20:06-04:00	#SriLanka surcharge on fuel removed!\n 🇨🇪 🇨🇪 \n The ...	negative
2	413231	2020-06-04 11:41:45-04:00	Net issuance increases to fund fiscal programs...	positive
4	830153	2020-07-09 10:39:14-04:00	\$AMD Ryzen 4000 desktop CPUs looking 'great' a...	positive
6	472959	2020-06-09 01:23:06-04:00	\$863.69 Million in Sales Expected for Spirit A...	positive
9	267894	2020-05-04 11:16:29-04:00	Where to Look for Dependable Dividends\n Read M...	neutral
...	...	...	...	...
928666	938666	2020-07-15 20:00:32-04:00	Real-Time Data Recovery Stalled Amid COVID-19 ...	NaN
928669	938669	2020-07-15 20:00:23-04:00	You \n \n \$SPX \$SPY #ES_F	NaN
928670	938670	2020-07-15 20:00:23-04:00	\$KO Coca-Cola #Options #maxpain Chart, Open In...	NaN
928671	938671	2020-07-15 20:00:06-04:00	Here's a dividends watchlist \n for the 01/04/0...	NaN
928672	938672	2020-07-15 20:00:00-04:00	\$AAL \$TWTR \$SPCE \$AZN \$ERI \$RCL \$JNJ \$AA \$SNBR...	NaN

577921 rows × 4 columns

Here we see that `retweets_df` has 350,752 rows and `remainder_df` has 577,921 rows. We should note that DataFrames are separated based on whether a tweet is a retweet or not. The number of rows in `retweets_df` and `remainder_df` gives an idea of the proportion of retweets in dataset but this doesn't directly tell us how many unique tweets were retweeted. A single original tweet can be retweeted many times by different users so that the 350,752 rows in `retweets_df` could represent a much smaller number of unique original tweets that were retweeted multiple times. We should also consider time delay effect based on typical Twitter behavior. It's highly probable that the majority of retweets in our dataset are from the same period but there would be some from slightly earlier period as the original tweets we collected.

1.4 Understanding distribution of tweets

Let's now try to understand the distribution of tweets over time. The following code takes the 'created_at' column as the index for resampling. It then resamples the data weekly and

calculates the count of tweets within each weekly interval. Finally, it counts the number of tweets that occurred within each week:

```
In [5]: # Get weekly tweet counts
weekly_counts = twitter_df.set_index('created_at').resample('W-MON', label='left',
weekly_counts = weekly_counts.rename_axis(index="Week Starting")
display(weekly_counts)
```

	id	tweet_text	sentiment
Week Starting			
2020-04-06 00:00:00-04:00	33291	33291	39
2020-04-13 00:00:00-04:00	90678	90678	143
2020-04-20 00:00:00-04:00	84788	84788	106
2020-04-27 00:00:00-04:00	52106	52106	71
2020-05-04 00:00:00-04:00	70438	70438	90
2020-05-11 00:00:00-04:00	0	0	0
2020-05-18 00:00:00-04:00	0	0	0
2020-05-25 00:00:00-04:00	35982	35982	53
2020-06-01 00:00:00-04:00	75666	75666	100
2020-06-08 00:00:00-04:00	73845	73845	98
2020-06-15 00:00:00-04:00	81982	81982	124
2020-06-22 00:00:00-04:00	90650	90650	122
2020-06-29 00:00:00-04:00	79637	79637	121
2020-07-06 00:00:00-04:00	93789	93789	129
2020-07-13 00:00:00-04:00	65821	65821	104

Here `.resample('W-MON', label='left', closed='left')` is the core of the operation. `.resample('W-MON')` resamples the data into weekly bins, starting on Mondays ('W-MON'). `label='left'` indicates that the label for each bin should be the left edge (the beginning) of the bin. `closed='left'` specifies that the left edge of each bin is inclusive (included in the bin). This means that tweets created on Monday will be included in the bin for that week.

This weekly distribution table provides opportunity to observe trends in tweet activity over time. Here we see that the tweet counts generally fluctuate but seem to be relatively high throughout June and July. And there were two weeks (May 11th and May 18th) with zero tweets recorded and possibly the following week (May 25th) also with reduced tweet activity. This could indicate a data collection issue or a period of low activity.

1.5 Visualising Twitter data

By considering the data volume and applying appropriate visualization techniques, we can gain more meaningful insights from the daily tweet count plot. Let's now group data by date and plot the daily tweet counts for the `twitter_df` DataFrame to investigate it. The following code first groups the `twitter_df` DataFrame by the date part of the 'created_at' column using `dt.date` to extract the date. This creates groups of tweets for each day. Then it counts the number of tweets in each group using `['tweet_text'].count()`, which counts the non-null values in the 'tweet_text' column. We then plot the data:

```
In [6]: # GroupBy date and count
daily_tweet_counts_twitter_all = twitter_df.groupby(twitter_df['created_at'].dt.date)

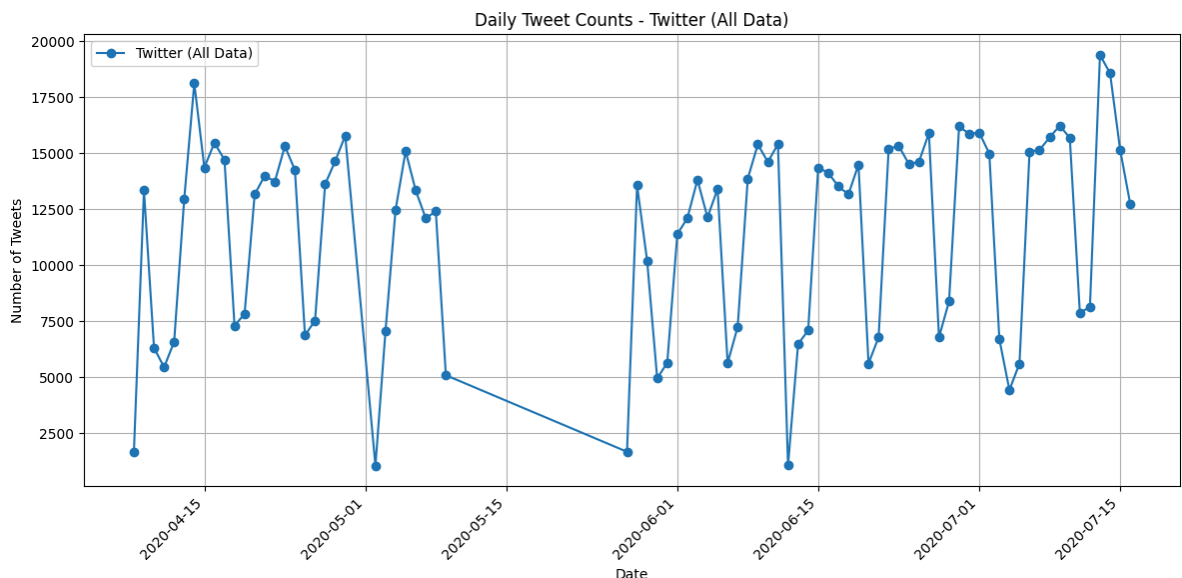
# Plotting
plt.figure(figsize=(12, 6))

plt.plot(daily_tweet_counts_twitter_all.index, daily_tweet_counts_twitter_all.value

plt.xlabel('Date')
plt.ylabel('Number of Tweets')
plt.title('Daily Tweet Counts - Twitter (All Data)')
plt.legend()
plt.grid(True)

plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability

plt.tight_layout() # Adjust layout to prevent labels from overlapping
plt.show()
```



By examining the plot, we are able to identify any gaps or periods where the tweet counts are significantly less or zero. These gaps might indicate missing data or periods where there was a decrease in tweet activity. If we need to further investigate specific gaps or patterns, we can adjust the date range of the plot or add annotations to highlight areas of interest. In

our case we know that there is gap for the period from May 10 to May 27 2020. This means there were either no tweets collected or recorded during that period, or they were filtered out during data processing.

There are various Data Imputation techniques to fill the missing data. For a simple approach, we could fill the gap with the average tweet count from the days before and after the gap. For more complex scenarios, we could explore time series imputation techniques or machine learning models to predict the missing values based on patterns in the existing data. However, this should be done cautiously and with awareness of the potential biases introduced by imputation. In our case neither of these techniques might be accurate since there is significant gap during which trends or events might be very unpredictable to infer or model. So we leave this gap as is for now and continue with further explorations.

1.6 Discovering cashtags and hashtags in Twitter data

In Twitter tweets, stock symbols can appear as either cashtags (e.g., `\$AAPL`, `\$GOOG`) or hashtags (e.g., `#AAPL`, `#GOOG`), both typically representing a company's stock ticker symbol. Cashtags are becoming more common, but hashtags are still used for broader stock-related discussions or events. These symbols can be placed anywhere within a tweet and often provide context for the message, linking it to a specific stock or topic.

Let's now uncover some insights into which stocks are being discussed most frequently in our Twitter data. The following code first extracts stock symbols from the beginning of each tweet in the `twitter_df` DataFrame using regular expressions and stores them in a new 'stock_symbols' column. Then, it explodes this column to create separate rows for each symbol in a tweet. It converts the symbols to lowercase, counts their occurrences, sorts them by frequency in descending order, and finally displays the top 10 most frequently mentioned stock symbols in the dataset:

```
In [7]: # Discover stock symbols and hashtags
twitter_df['cashtags_hashtags'] = twitter_df['tweet_text'].str.findall(r'(?:\$|#)([a-zA-Z]+)')
twitter_df.explode('cashtags_hashtags')['cashtags_hashtags'].str.lower().value_counts()
```

```
Out[7]: cashtags_hashtags
stocks      262381
spx         190857
aapl        123629
spy         115031
amzn        108484
fb           85685
es           66068
stockmarket  65091
msft         62018
trading      60261
Name: count, dtype: int64
```

Here `.str.findall(r'(?:\$|#)([a-zA-Z]+)')` applies a regular expression to find and extract cashtags and hashtags from the tweet text. Let's break down

the regex:

- `(?: ... )` : This creates a non-capturing group. It means this part of the regex is used for matching, but the matched content won't be separately captured or stored.
- `\$|#` : This matches either a literal dollar sign (\$) or a hashtag (#). The `|` symbol represents an "or" condition.
- `[a-zA-Z]+` : Matches one or more letters (a-z, A-Z).
- `\.*` : Matches zero or one dot (.), allowing for symbols with dots (e.g., \$BRK.B).
- `[a-zA-Z]+` : Matches one or more letters again, capturing the rest of the symbol after the dot (if any).

So the entire regex in the first line in the above code searches for patterns that look like stock symbol cashtags (e.g., \$AAPL, \$AAPL, \$GOOG, \$BRK.B) or hashtags and stores them in a new column called 'cashtags_hashtags'.

`twitter_df.explode('cashtags_hashtags')` in the second line of the above code "explodes" the 'cashtags_hashtags' column, transforming each list of cash-/hashtags into separate rows. So, if a tweet had multiple cash-/hashtags, it will now have multiple rows, each with one tag. Then the remainder of second line in the above code takes the extracted cash-/hashtags, counts how many times each tag appears in the tweets, sorts them by frequency, and shows the top 10 most mentioned cash-/hashtags.

Overall, this table highlights that the S&P 500 index (spx) together with SPDR S&P 500 ETF Trust (spy) and E-mini S&P 500 futures (es) (both are financial instruments that track the performance of the S&P 500 index) are among the top 10 mentions in this Twitter dataset. This is not surprising since this Twitter dataset itself was originally collected using S&P 500 tag and other top 25 companies in the S&P 500 index. From the above table we can also gather that other most discussed financial instruments within the analyzed Twitter data are Apple (aapl), Amazon (amzn), FB (fb), and Microsoft (msft). Hashtags "stocks", "stockmarket" and "trading" further emphasize the dataset's focus on stock market-related topics. This information can be valuable for understanding market trends, investor sentiment, and the overall focus of financial discussions on Twitter during the period covered by the dataset.

1.7 Filtering Twitter data

Let's now try to filter some of tweets for specific terms. The following code defines a function called `contains_spy()` that checks if a given text string contains any mention of the SPY ETF (SPDR S&P 500 ETF Trust), considering various textual variations. This time we check entire tweet text instead of just searching for cashtags and hashtags. First we use `text.lower()` which converts the input text to lowercase to ensure case-insensitive matching. `patterns = [...]` defines a list of regular expression patterns representing different ways the SPY ETF might be mentioned. It includes variations like "spy", "\$spy", "#spy", "spydr", "s&p 500 etf", with and without word boundaries (`\b`). Word boundaries ensure that "spy" is not matched within words like "spying". In essence, this function aims to

identify text entries in tweets that mention the SPY ETF, regardless of how it is written. Then the code filters the `twitter_df` DataFrame to keep only the tweets that mention the SPY ETF, using the `contains_spy()` function:

```
In [8]: # Define a function to check for S&P 500 mentions with variations
def contains_spy(text):
    # Handle variations in spacing, "&", and case
    text = text.lower() # Convert to lowercase for case-insensitive matching
    text = re.sub(r"[^a-zA-Z0-9 ]", "", text) # Remove special characters except s

    # Check for different patterns
    patterns = [
        r"\bsp\b", # Using word boundaries (\b) to avoid capturing part of word (
        r"\$spy", # Cashtag
        r"#spy", # Hashtag
        r"\b\$spy\b", # Cashtag with word boundaries
        r"\b#spy\b", # Hashtag with word boundaries
        r"\bspdr\b", # Full name part using word boundaries
        r"\bs&p 500 etf\b", # Full name part with variations using word boundarie
        r"\bs&p500 etf\b", # Full name part with variations using word boundaries
        r"\bsp500 etf\b" # Full name part with variations using word boundaries
        r"\bsp etf\b" # Full name part with variations using word boundaries
        # ...add more variations as needed...
    ]

    return any(re.search(pattern, text) for pattern in patterns)

# Apply the function to filter the DataFrame and display it
filtered_twitter_df = twitter_df[twitter_df['tweet_text'].apply(contains_spy)]
filtered_twitter_df
```

Out[8]:

	id	created_at	tweet_text	sentiment	cashtags_hashtags
32	36981	2020-04-13 13:31:40-04:00	TOS frozen for anyone else? \$es_f \$spx \$spy...	neutral	[es, spx, spy, options, futures, cl, forex]
43	145407	2020-04-21 09:20:35-04:00	\$ES_F Needs to trade \$90s+ or \$2640-\$2650 on d...	neutral	[ES, SPY, SPX]
63	472181	2020-06-09 03:44:58-04:00	Remember the turn? Now start preparing for the...	negative	[TSLA, SPY, GILD, ABBV, PFE, TEVA, TDOC, VIX, ...]
65	22115	2020-04-11 15:22:03-04:00	#ES_F #CL_F #NQ_F \$GOLD #OOTT #OATT \$SPY \$SPX ...	neutral	[ES, CL, NQ, GOLD, OOTT, OATT, SPY, SPX, Tradi...]
78	530059	2020-06-15 08:45:55-04:00	Only in America do people believe that buying ...	positive	[SPX, SPY]
...	...	...	...	...	...
928632	938631	2020-07-15 20:04:42-04:00	RT @WarlusTrades: \$SPX SPY #ES_F #AMD Co...	NaN	[SPX, ES, AMD]
928639	938639	2020-07-15 20:03:56-04:00	@idekthepoint sorry Dinky its too much for you...	NaN	[SPX, SPY]
928652	938652	2020-07-15 20:02:21-04:00	lows. This is shaping up for another nice sell...	NaN	[economy, SPX, NDX, LVGO, DDOG, CRWD, TSLA, SP...]
928662	938662	2020-07-15 20:00:57-04:00	RT @WarlusTrades: \$SPX SPY #ES_F #AMD Co...	NaN	[SPX, ES, AMD]
928669	938669	2020-07-15 20:00:23-04:00	You \$SPX \$SPY #ES_F	NaN	[SPX, SPY, ES]

99246 rows × 5 columns

The resulting `filtered_twitter_df` DataFrame contains a subset of the original data, specifically focusing on tweets relevant to the SPY ETF. This filtering step is essential for further analysis or visualization tasks when we are specifically interested in tweets related to the SPY ETF.

2 What is StockTwits data

StockTwits is a social media platform specifically designed for investors and traders to share ideas and information about the stock market. It's like Twitter, but with a focus on financial discussions. StockTwits aims to provide a platform for investors of all levels to connect, share ideas, and stay informed about the stock market in real time. It's a valuable tool for understanding market sentiment, discovering new investment ideas, and engaging with a community of like-minded individuals.

StockTwits fosters a strong sense of community by enabling users to connect with each other, follow interesting profiles, and engage in discussions about specific stocks or broader market trends. It's a space for collaborative learning and idea exchange. It offers ability to engage in Stock-Specific Discussions. Conversations are often organized using "cashtags" (similar to hashtags on Twitter), which are denoted by a dollar sign followed by a stock symbol (e.g., \$AAPL for Apple stock). This feature makes it easy to find discussions and sentiment related to particular companies.

StockTwits' distinct feature is to allow users to express their attitude and overall emotional tone or opinion regarding a particular stock, market trend, or investment strategy. It's essentially a measure of whether people are feeling bullish (positive), bearish (negative), or neutral about a specific topic. On the StockTwits platform, users can tag their messages as either Bearish or Bullish to indicate their sentiment or outlook on a particular stock or the market in general. These tags provide a quick and visual way for other users to understand the sentiment behind a message. These tags can be leveraged for:

- **Sentiment Analysis:** The Bearish and Bullish tags provide valuable data for sentiment analysis algorithms. These algorithms can use the tags to quickly identify and quantify the overall sentiment expressed by users towards specific stocks or the market as a whole.
- **Filtering and Search:** Users can filter their StockTwits feed or search for messages based on these tags. This allows them to focus on messages that align with their own sentiment or to get a sense of the prevailing sentiment around a particular stock.
- **Community Engagement:** The tags encourage users to explicitly express their opinions and engage in discussions with others who share similar or opposing views. This fosters a more dynamic and interactive community.

Important considerations for Bearish and Bullish tags on StockTwits:

- **Subjectivity:** The Bearish and Bullish tags reflect the individual opinions of the users who post them. These opinions might be based on different factors, including personal biases, research, or speculation. It's important to remember that these tags are subjective and might not always be accurate.
- **Noise and Misinformation:** Social media platforms like StockTwits can also contain a significant amount of noise and misinformation. There's always a possibility that some users might intentionally try to manipulate sentiment by posting misleading or exaggerated messages with certain tags. It's advisable to be cautious and to verify

information from multiple sources before making any investment decisions based on sentiment data. It's crucial to verify information from multiple sources and to avoid relying solely on sentiment data for investment decisions.

- **Dynamic Nature of Sentiment:** Sentiment can change rapidly, especially in response to market events or news. It's essential to monitor sentiment trends over time rather than relying on a single snapshot.

In summary, the Bearish and Bullish tags on StockTwits provide a valuable tool for understanding market sentiment and engaging with the financial community. However, it's important to use them wisely and in conjunction with other forms of analysis to make informed investment decisions.

2.1 Getting StockTwits data

Unfortunately, StockTwits no longer offers a publicly available developer API or developer accounts for general use. StockTwits did have a developer API program in the past, but it have been significantly restricted recently. The previous API allowed developers to access data like streams, sentiment, symbols, and user information. Currently available data access options are via Enterprise Partnerships, Third-Party Data Providers, or Public Datasets.

In our lesson we explore 'SPY Messages Data from Stocktwits' dataset hosted on figshare and available at <https://doi.org/10.6084/m9.figshare.20237736.v1>. Figshare is a repository for research data, including datasets, figures, and other research outputs. It encourages researchers to share their data openly and allows users to upload and contribute datasets.

This dataset is also available under the Creative Commons Attribution 4.0 International (CC BY 4.0) license. This dataset is provided in CSV format and contains a collection of messages posted on the StockTwits platform related to the SPY ETF (SPDR S&P 500 ETF Trust).

The following code downloads this dataset from figshare, saves it as a CSV file, and then reads the data into `stocktwits_df` pandas DataFrame for further analysis:

```
In [9]: # Define the chunk size (adjust this value as needed)
        chunksize = 10000

        # Create an empty list to store processed chunks
        processed_chunks = []

        # Iterate through the CSV file in chunks
        for chunk in pd.read_csv("SPY_stocktwits_messages.csv", chunksize=chunksize):
            processed_chunks.append(chunk) # Append the chunk to the list

        # Concatenate the processed chunks into a single DataFrame named stocktwits_df
        stocktwits_df = pd.concat(processed_chunks, ignore_index=True)
        stocktwits_df
```

Out[9]:

	Unnamed: 0	Sentence	Timestamp	DateTime	Sentiment
0	25	\$SPY watching squawk box im losing brain cells	1584964231000	2020/03/23 07:50:31	NaN
1	24	\$SPY there's no cure	1584964276000	2020/03/23 07:51:16	Bearish
2	22	\$SPY \$DJIA \$AAPL \$TSLA \n\nlt's only just begi...	1584964278000	2020/03/23 07:51:18	NaN
3	21	\$SPY uh oh	1584964308000	2020/03/23 07:51:48	Bullish
4	20	\$TVIX Im in a NY hospital with my mom. They do...	1584964331000	2020/03/23 07:52:11	NaN
...	...	...	...	...	...
3261862	2135962	\$SPY The sinking love boat sailing south!	1649232079000	2022/04/06 04:01:19	NaN
3261863	2135961	\$SPY	1649232096000	2022/04/06 04:01:36	Bearish
3261864	2135960	\$SPY \$QQQ Expecting more downside or are we at...	1649232111000	2022/04/06 04:01:51	NaN
3261865	2135959	\$SPY \$QQQ already recovering now US markets open	1649232116000	2022/04/06 04:01:56	Bullish
3261866	2135958	\$SPY that sell off was a mistake watch. Some ...	1649232117000	2022/04/06 04:01:57	NaN

3261867 rows × 5 columns

Please note that size of original 'SPY Messages Data from Stocktwits' dataset hosted on figshare is about 452.52 MB. When dealing with large datasets, like the StockTwits dataset, attempting to load the entire file into memory at once can lead to memory exhaustion and crashes. In our code we download this dataset in chunks to prevent overloading the memory. By reading the data in smaller chunks, we process the dataset piece-by-piece, reducing the memory footprint at any given time. This allows to work with larger-than-memory datasets without running into memory issues.

2.2 Adjusting time zone in StockTwits data

The StockTwits dataset covers much longer period spanning 2 years from 2020 to 2022 than our Twitter dataset. The following code focuses on filtering the StockTwits messages

DataFrame (`stocktwits_df`) to include only messages within a specific date range, from April 9, 2020, to July 16, 2020 that corresponds to data availability window for Twitter dataset in previous section of this lesson.

First, we again convert date values to DateTime object. `stocktwits_df['DateTime'] = pd.to_datetime(stocktwits_df['DateTime'])` line converts values in the 'DateTime' column in the `stocktwits_df` DataFrame to pandas DateTime objects. This is essential for performing date-based filtering and comparisons effectively. Then we define the start and end dates of the desired date range and filter the original DataFrame to create a new DataFrame containing only messages posted between April 9, 2020, and July 16, 2020, inclusive:

```
In [10]: # Convert 'DateTime' column to DateTime objects
stocktwits_df['DateTime'] = pd.to_datetime(stocktwits_df['DateTime'])

# Filter stocktwits_df DateTime from April 9 to July 16, 2020
start_date = pd.to_datetime('2020-04-09')
end_date = pd.to_datetime('2020-07-16')

filtered_stocktwits_df = stocktwits_df[(stocktwits_df['DateTime'] >= start_date) &
filtered_stocktwits_df
```

Out[10]:

	Unnamed: 0	Sentence	Timestamp	DateTime	Sentiment
151678	168399	\$SPY I currently have \$AAPL, \$XOM, and \$JBLU C...	1586404800000	2020-04-09 00:00:00	NaN
151679	168398	\$SPY So besides NYC the rest of the world is b...	1586404834000	2020-04-09 00:00:34	Bearish
151680	168397	\$SPY bulls thinking numbers are priced in beca...	1586404839000	2020-04-09 00:00:39	Bearish
151681	168396	\$SPY plunge protection team is in full effect!	1586404863000	2020-04-09 00:01:03	Bullish
151682	168395	\$SPY if i played futures i'd go long right...	1586404897000	2020-04-09 00:01:37	NaN
...	...	...	...	...	...
716261	775402	\$SPY y'all know what tomorrow is. Jobs.	1594871738000	2020-07-15 23:55:38	Bullish
716262	775401	\$SPY TMRW WILL BE GREAT AGAIN	1594871761000	2020-07-15 23:56:01	Bullish
716263	775400	\$SPY LMAO: "The center said the Endangere...	1594871825000	2020-07-15 23:57:05	NaN
716264	775399	\$SPY here it comes, the pump BECUASE you know ...	1594871837000	2020-07-15 23:57:17	Bearish
716265	775398	\$QQQ \$SPY Futures bleeding	1594871956000	2020-07-15 23:59:16	NaN

564588 rows × 5 columns

In this DataFrame we have both 'Timestamp' column and 'DateTime' column. It's common for 'Timestamp' columns to store Unix timestamps, which represent the number of seconds that have elapsed since January 1, 1970, at 00:00:00 Coordinated Universal Time (UTC). This is a standard way to represent time in many systems. Other options are Epoch Time (the number of seconds since a specific reference point), Relative Time (the time elapsed since the start of a specific event or process), or some other custom time formats. There are many ways to investigate and work with Timestamps. As a startig point in our case let's convert 'Timestamp' column to Unix timestamps, but in milliseconds and compare the newly created 'DateTime_from_Timestamp' column with the existing 'DateTime' column to see if they represent the same information or if there are any differences.

```
In [11]: # Convert Timestamp values to pandas DateTime object
filtered_stocktwits_df.loc[:, 'DateTime_from_Timestamp'] = pd.to_datetime(filtered_
```

```
filtered_stocktwits_df
```


Out[11]:

	Unnamed: 0	Sentence	Timestamp	DateTime	Sentiment	DateTime_from_Tim
151678	168399	\$SPY I currently have \$AAPL, \$XOM, and \$JBLU c...	1586404800000	2020-04-09 00:00:00	NaN	2020-04-09 C
151679	168398	\$SPY So besides NYC the rest of the world is b...	1586404834000	2020-04-09 00:00:34	Bearish	2020-04-09 C
151680	168397	\$SPY bulls thinking numbers are priced in beca...	1586404839000	2020-04-09 00:00:39	Bearish	2020-04-09 C
151681	168396	\$SPY plunge protection team is in full effect!	1586404863000	2020-04-09 00:01:03	Bullish	2020-04-09 C
151682	168395	\$SPY if i played futures i'd go long right...	1586404897000	2020-04-09 00:01:37	NaN	2020-04-09 C
...	...	...	...	...	...	...
716261	775402	\$SPY y'all know what tomorrow is. Jobs.	1594871738000	2020-07-15 23:55:38	Bullish	2020-07-16 C
716262	775401	\$SPY TMRW WILL BE GREAT AGAIN	1594871761000	2020-07-15 23:56:01	Bullish	2020-07-16 C
716263	775400	\$SPY LMAO: "The center said the Endangere...	1594871825000	2020-07-15 23:57:05	NaN	2020-07-16 C
716264	775399	\$SPY here it comes, the pump BECUASE you know ...	1594871837000	2020-07-15 23:57:17	Bearish	2020-07-16 C

	Unnamed: 0	Sentence	Timestamp	DateTime	Sentiment	DateTime_from_Tim
716265	775398	\$QQQ \$SPY Futures bleeding	1594871956000	2020-07- 15 23:59:16	NaN	2020-07-16 (

564588 rows × 6 columns

Ideally both 'Timestamp' and 'DateTime' columns are supposed to represent the same time information. However, here we see that time difference between the newly created 'DateTime_from_Timestamp' column with the existing 'DateTime' column is consistently 4 hours. This suggests that correct timezone should be New York timezone for this dataset. The following code transforms the timestamp data into a more usable and timezone-aware format:

```
In [12]: # Convert and Localize Timestamp values to NY timezone
filtered_stocktwits_df['DateTime_from_Timestamp'] = pd.to_datetime(filtered_stocktwits_df['Timestamp']).tz_localize('America/New_York')
```

Out[12]:

	Unnamed: 0	Sentence	Timestamp	DateTime	Sentiment	DateTime_from_Tim
151678	168399	\$SPY I currently have \$AAPL, \$XOM, and \$JBLU c...	1586404800000	2020-04-09 00:00:00	NaN	2020-04-09 00:00:0
151679	168398	\$SPY So besides NYC the rest of the world is b...	1586404834000	2020-04-09 00:00:34	Bearish	2020-04-09 00:00:3
151680	168397	\$SPY bulls thinking numbers are priced in beca...	1586404839000	2020-04-09 00:00:39	Bearish	2020-04-09 00:00:3
151681	168396	\$SPY plunge protection team is in full effect!	1586404863000	2020-04-09 00:01:03	Bullish	2020-04-09 00:01:0
151682	168395	\$SPY if i played futures i'd go long right...	1586404897000	2020-04-09 00:01:37	NaN	2020-04-09 00:01:3
...	...	...	...	...	...	...
716261	775402	\$SPY y'all know what tomorrow is. Jobs.	1594871738000	2020-07-15 23:55:38	Bullish	2020-07-15 23:55:3
716262	775401	\$SPY TMRW WILL BE GREAT AGAIN	1594871761000	2020-07-15 23:56:01	Bullish	2020-07-15 23:56:0
716263	775400	\$SPY LMAO: "The center said the Endangere...	1594871825000	2020-07-15 23:57:05	NaN	2020-07-15 23:57:0
716264	775399	\$SPY here it comes, the pump BECUASE you know ...	1594871837000	2020-07-15 23:57:17	Bearish	2020-07-15 23:57:1

	Unnamed: 0	Sentence	Timestamp	DateTime	Sentiment	DateTime_from_Tim
716265	775398	\$QQQ \$SPY Futures bleeding	1594871956000	2020-07- 15 23:59:16	NaN	2020-07-15 23:59:1

564588 rows × 6 columns

By localizing and converting the timestamps, we ensure that the datetime values in the DataFrame are timezone-aware and represent the correct local time in New York. This is crucial for accurate analysis and interpretation of time-based data. This process of timezone localization and conversion is essential for consistency in datetime representation when working with time-based data, particularly when dealing with data from different sources or locations, to ensure accuracy and consistency in analysis.

2.3 Visualising StockTwits data

Let's now craft the code to plot the `filtered_stocktwits_df` data as a bar plot, grouped by date and showing the counts of 'Bearish', 'Neutral', and 'Bullish' values in the Sentiment column. The following code processes the `filtered_stocktwits_df` DataFrame to replace NaN values in the 'Sentiment' column with 'Neutral', groups the data by date and sentiment, and then creates an interactive bar plot using Plotly with specific color mapping for different sentiments.

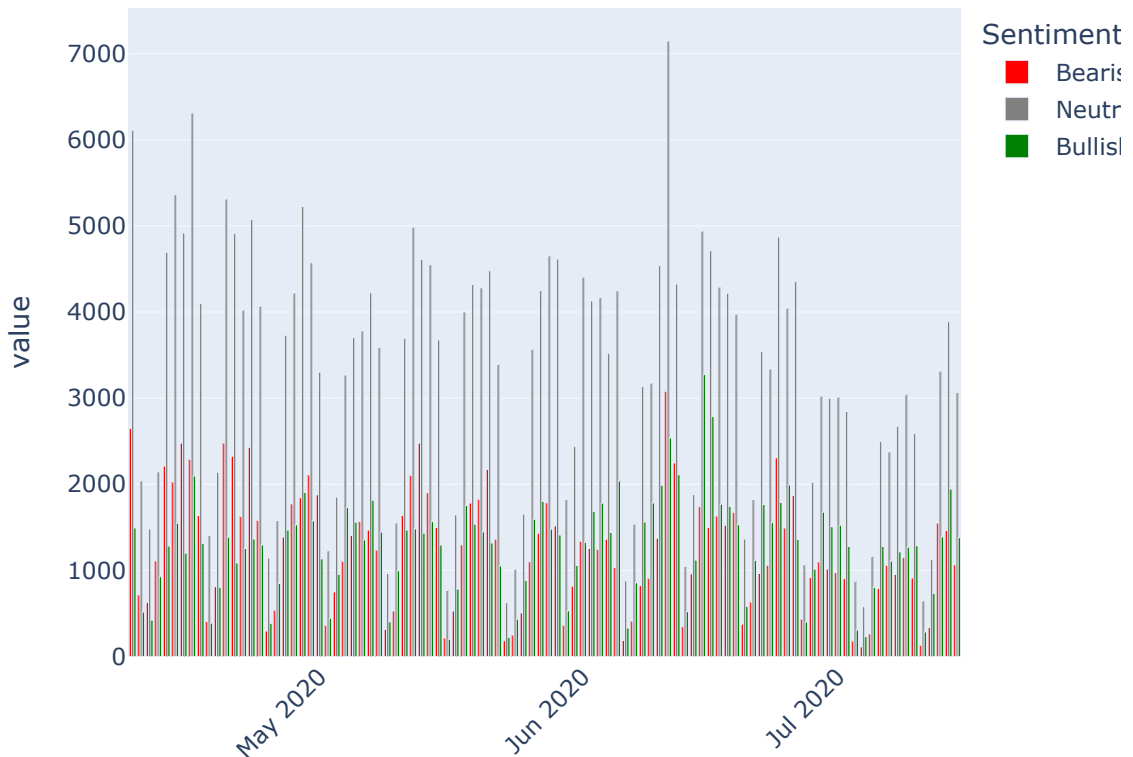
```
In [13]: # Replace 'NaN' with 'Neutral' in the 'Sentiment' column before grouping, using .loc
filtered_stocktwits_df.loc[filtered_stocktwits_df['Sentiment'].isnull(), 'Sentiment'] = 'Neutral'

# Group the data by date and sentiment, then count occurrences
sentiment_counts = filtered_stocktwits_df.groupby(['DateTime', 'Sentiment']).count()
sentiment_counts = sentiment_counts[['Bearish', 'Neutral', 'Bullish']] # reorder columns

# Create the interactive bar plot using Plotly
fig = px.bar(sentiment_counts, x=sentiment_counts.index, y=sentiment_counts.columns,
             labels={'x': 'Date', 'y': 'Count'},
             title='Daily Sentiment Counts - StockTwits (Bearish, Neutral, Bullish)',
             color_discrete_map={'Bearish': 'red', 'Neutral': 'grey', 'Bullish': 'green'})

# Customize the layout for better readability and display the plot
fig.update_layout(barmode='group', xaxis_tickangle=-45)
fig.show()
```

Daily Sentiment Counts - StockTwits (Bearish, Neutral, Bullish)



This approach allows to visualize the daily sentiment trends on StockTwits, with clear color-coded representation for Bearish, Neutral, and Bullish sentiments, enhancing the interpretability of the plot. The bar plot visualizes daily sentiment counts for StockTwits messages about the SPY ETF, categorized as Bearish (red), Neutral (grey), or Bullish (green). The x-axis represent dates, while the y-axis represent message counts. Three bars per date show counts for each sentiment. Plotly's interactivity enables zooming, hovering for details, and potential data selection, allowing in-depth exploration of sentiment trends over time.

2.4 Discovering stock symbols in StockTwits data

StockTwits messages often contain stock symbols, also known as tickers or cashtags. Users frequently include them to denote the specific stocks they are discussing. These symbols are typically represented by a dollar sign followed by the stock's ticker symbol (e.g., `\$AAPL` for Apple, `\$SPY` for SPDR S&P 500 ETF Trust).

Similar to how we did for Twitter dataset here again we can extract stock symbols from StockTwits messages. The following code aims to extract stock symbols (tickers) from the 'Sentence' column of the `filtered_stocktwits_df` DataFrame, count their occurrences, and display the top 10 most frequently mentioned tickers:

```
In [14]: # Discovering stock symbols in StockTwits data
filtered_stocktwits_df['tickers'] = filtered_stocktwits_df['Sentence'].str.findall(
filtered_stocktwits_df.explode(['tickers'])['tickers'].value_counts().sort_values(a
```

```
Out[14]: tickers
$SPY      553357
$QQQ      27869
$spy      12975
$SPX      11136
$AAPL     10631
$DJIA      8977
$DIA       8949
$ES        8062
$TSLA      7627
$BA        5368
Name: count, dtype: int64
```

This table shows the top 10 most frequently mentioned stock symbols (tickers) in the StockTwits dataset, along with their respective counts. The most prominent observation to be made is SPY Dominance - the ticker `\$SPY` (SPDR S&P 500 ETF Trust) and `\$spy` (its lowercase version) is by far the most mentioned stock symbol in the dataset, appearing over 553,000 times. This is not surprising considering that this entire StockTwits dataset is SPY centric.

The table reveals that the StockTwits dataset primarily focuses on discussions about the S&P 500 index and related ETFs. However, there is also significant interest in other major market indices, individual stocks (particularly large-cap companies), and futures contracts. This information provides insights into the topics and trends that are most prevalent within the StockTwits community during the analyzed period.

2.5 Comparing Twitter and StockTwits data

Let's now compare Twitter and StockTwits data. The following code aims to do this by creating a line plot. First we calculate the daily tweet counts from the `filtered_twitter_df` DataFrame by grouping the data by the date part of the 'created_at' column (using `dt.date` to extract the date) and then counting the number of tweets in each group by counting the non-null values in the 'tweet_text' column. We do the similar operation for the `filtered_stocktwits_df` DataFrame, using the 'DateTime_from_Timestamp' column for date grouping and the 'Sentence' column for counting messages. Then we create and customize plot:

```
In [15]: # Group Twitter data by date and count tweets
daily_tweet_counts_twitter = filtered_twitter_df.groupby(filtered_twitter_df['creat

# Group StockTwits data by date and count messages
daily_tweet_counts_stocktwits = filtered_stocktwits_df.groupby(filtered_stocktwits_

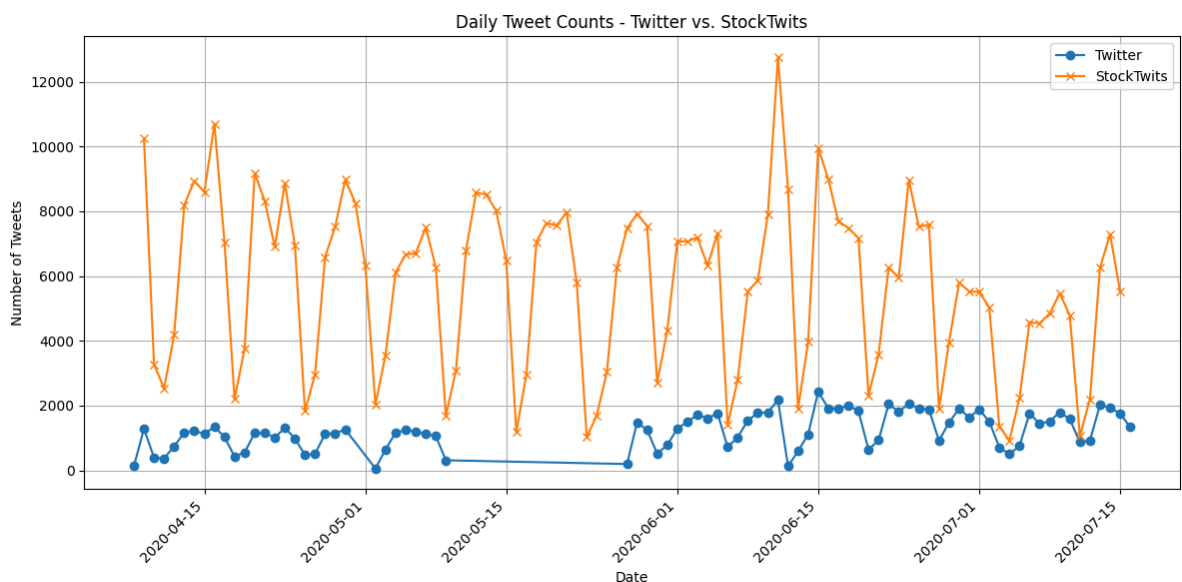
# # Create a figure and plot Twitter and StockTwits data
```

```
plt.figure(figsize=(12, 6))
plt.plot(daily_tweet_counts_twitter.index, daily_tweet_counts_twitter.values, label='Twitter')
plt.plot(daily_tweet_counts_stocktwits.index, daily_tweet_counts_stocktwits.values, label='StockTwits')

# Customize the plot
plt.xlabel('Date')
plt.ylabel('Number of Tweets')
plt.title('Daily Tweet Counts - Twitter vs. StockTwits')
plt.legend()
plt.grid(True)

# Rotate x-axis labels for better readability
plt.xticks(rotation=45, ha='right')

# Adjust layout to prevent overlapping labels and display the plot
plt.tight_layout()
plt.show()
```



First let's observe that for the same time period `filtered_twitter_df` has 99,246 rows, while `filtered_stocktwits_df` has 564,588 rows. This information provides valuable context for interpreting the daily tweet count plots. It highlights the significant difference in the volume of data between the two platforms for the selected period. Here are some additional points to keep in mind:

- **Platform Activity:** The difference in row counts suggests that StockTwits might have had much higher activity or user engagement compared to Twitter for the given period and filtering criteria. This could be due to various factors, including the platforms' user base, focus, and popularity within the investment community.
- **Data Collection and Integrity:** The difference in data volume might be influenced by the data collection methods used. Data collectors use different APIs or data sources for Twitter and StockTwits and there might be variations in how data was collected, filtered, or processed, leading to differences in the final row counts. We also need to ensure that

the data collection and filtering processes were consistent and reliable for both platforms to minimize potential biases or errors in the analysis.

- **Analysis Implications:** When comparing trends or patterns between Twitter and StockTwits, keep in mind that the absolute tweet counts might not be directly comparable due to the difference in data volume. We need to consider using normalization or relative metrics to make more meaningful comparisons.
- **Data Gap Context:** While the gap in Twitter data from May 10 to May 27 2020 is important to acknowledge, it's also essential to consider it in the context of the overall data volume. If the gap represents a small proportion of the total Twitter data, its impact on the overall analysis might be limited. However, if the gap is significant relative to the total data, it could affect the interpretation of trends or patterns.

By considering the overall data volume difference and its potential implications, we can make more informed interpretations of the daily tweet count plots and draw more robust conclusions from analysis. We need to remember to clearly communicate the data volume difference and any normalization or scaling applied in reports or presentations for transparency.

X. Conclusion

In this lesson we explored applications of alternative data. Alternative Data in finance refers to non-traditional data sources used to gain insights into market trends, asset pricing, and investment decisions. This contrasts with traditional data like financial statements, company filings, and market prices. Text data from social media falls under this category as it reflects public sentiment, market chatter, and news dissemination that can influence asset prices. Specifically, Twitter and StockTwits data are considered valuable sources of alternative data in financial engineering due to their real-time nature and focus on financial discussions and stock market activity.

In this lesson we explored and compared Twitter and StockTwits data to see how they could be used for financial analysis. Main focus was on the SPY ETF (SPDR S&P 500 ETF Trust) and a specific time period. This involved getting data from both Twitter and StockTwits, adjusting timezones, and handling retweets on Twitter; Creating plots to see how many tweets were posted each day on each platform to understand the differences in data volume and user activity.

References

- Bruno Taborda, Ana de Almeida, José Carlos Dias, Fernando Batista, Ricardo Ribeiro, April 15, 2021, "Stock Market Tweets Data", IEEE Dataport, doi: <https://dx.doi.org/10.21227/g8vy-5w61>

- Liu, Jin-Xian (2022). SPY_stocktwits_messages.csv. figshare. Dataset.
<https://doi.org/10.6084/m9.figshare.20237736.v1>

Copyright 2024 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.

Lesson 4 - Social Media

December 25, 2025

MODULE 4 | LESSON 4

1 Social Media

Reading Time	6h
Prior Knowledge	API, function of social media in general
Keywords	API, youtube, reddit, subreddit, sentiment analysis, nltk, keywords

Social media has become an integral part of modern life, transforming how we communicate, share information, and interact with the world around us. To understand the significance of social media data and its applications today, it's essential to examine the historical evolution of these platforms.

The roots of social media can be traced back to the early days of the internet. In 1997, SixDegrees.com launched as the first recognizable social network site, allowing users to create profiles and list friends. This paved the way for subsequent platforms like Friendster, MySpace, and eventually Facebook, which revolutionized the social media landscape. As these platforms grew in popularity and functionality, they began generating vast amounts of user data. This data, encompassing everything from user profiles and connections to content interactions and behavioral patterns, quickly became a valuable resource for researchers, marketers, and businesses.

The rise of social media data as a field of study and application coincided with the increasing sophistication of these platforms. Facebook's introduction of the News Feed in 2006, for instance, not only changed how users interacted with the platform but also provided a rich source of data on user preferences and behaviors. Today, social media data is used across various domains, including marketing, public health, political science, and social research. Platforms like YouTube and Reddit offer unique insights into user behavior, content consumption, and public discourse. Through APIs (application programming interfaces) provided by these platforms, researchers and developers can access and analyze this data in unprecedented ways.

In this lesson, we will focus on the practical aspects of retrieving and working with social media data. We'll explore how to use APIs to collect data from YouTube and Reddit, demonstrating the real-world applications of social media data analysis. By understanding both the historical context and current methodologies, we can better appreciate the power and potential of social media data in today's digital landscape.

1.1 1. What is Social Media?

We will provide some definitions as to what social media is even though each and every one of us probably has enough exposure to such platforms.

Boyd and Ellison define social network sites as web-based services that allow individuals to * construct a public or semi-public profile within a bounded system, * articulate a list of other users with whom they share a connection, and * view and traverse their list of connections and those made by others within the system. This information highlights the structural aspects of social media, such as user profiles, connections, and network visibility.

The Merriam-Webster's Dictionary definition gives emphasis to the communication and community building aspects of social media: the social networks are forms of electronic communication (such as websites for social networking and microblogging) through which users create online communities to share information, ideas, personal messages, and other content (such as videos) ("Social media" [*Merriam-Webster.com Dictionary*]).

On the other hand, Cambridge Dictionary describes social media as technological platforms: websites and computer programs that allow people to communicate and share information on the internet using a computer or mobile phone ("Social media" [*Cambridge Dictionary*]).

In short, we can sum up the aspects of social media as follows: * User profiles and connections * Online communities * Information and content sharing * Technological platforms * User-generated content * Interactivity and collaboration * Virtual networks * Expression and communication

Each of the above characteristics is a data-generating process. For example, the "User profiles and connections" part of social media generates demographic data, the social network itself (friends, followers, and connections), and personal interests and preferences. The "Online communities" generate group membership data, community interaction patterns, and interests/trends within different communities.

It should be clear by now that the vast amount of data generated can be used for multiple purposes that depend on the objectives of the researcher. Specifically, in the context of finance, we are very interested in the "User-generated content" from which we can extract: * text data: posts, comments * sentiment and opinion data * multimedia content related to financial topics

This data is valuable for several finance-specific applications:

- Market sentiment analysis: By analyzing the sentiment of social media posts about specific stocks, commodities, market conditions, etc., researchers can quantify public opinion.
- Stock price prediction: Some studies have shown correlations between social media sentiment and short-term stock price changes, making this data potentially useful for trading strategies.
- Financial news dissemination: Social media often spreads financial news faster than traditional media!
- Risk assessment: By monitoring social media discussions, financial institutions can identify potential risks or reputational issues early.
- Cryptocurrency trends: Given the significant role of online communities in cryptocurrency markets, social media data is particularly valuable in this sector.

1.2 2. Retrieving the Data

In this section, we will provide several examples of how to use an API in order to connect to a social media platform, how to retrieve the data, and how the data looks like before structuring it into dataframes for plotting and analysis.

1.2.1 2.1 An Example of Quantifying Sentiment Using YouTube Data

In this subsection, we'll explore how to leverage YouTube API to gather and analyze public sentiment around significant financial events. We'll use the 2020 market crash as a case study to understand how sentiment can be quantified and correlated with financial data like the S&P 500 index.

Pre-requisites:

- Google Account: Ensure you have an active Google account. This is necessary to access Google's API services.
- Google Cloud Project: You'll need to create a project in the Google Cloud Console. This project will host your API key, which is essential for accessing YouTube's data.

YouTube API Setup: Follow these steps to get started:

1. Go to the Google Cloud Console.
2. Create a new project.
3. Navigate to the API Library and enable the YouTube Data API v3.
4. Create credentials (API key) for accessing the API.

```
[1]: !pip install pip --upgrade
!pip install yfinance --upgrade
```

```
Requirement already satisfied: pip in /usr/local/lib/python3.11/site-packages
(25.3)
```

```
WARNING: Running pip as the 'root' user can result in broken permissions
and conflicting behaviour with the system package manager, possibly rendering
your system unusable. It is recommended to use a virtual environment instead:
https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know
what you are doing and want to suppress this warning.
```

```
Requirement already satisfied: yfinance in /usr/local/lib/python3.11/site-
packages (1.0)
```

```
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/site-
packages (from yfinance) (2.0.0)
```

```
Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.11/site-
packages (from yfinance) (1.26.2)
```

```
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.11/site-
packages (from yfinance) (2.32.3)
```

```
Requirement already satisfied: multitasking>=0.0.7 in
```

```

/usr/local/lib/python3.11/site-packages (from yfinance) (0.0.11)
Requirement already satisfied: platformdirs>=2.0.0 in
/usr/local/lib/python3.11/site-packages (from yfinance) (3.4.0)
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.11/site-
packages (from yfinance) (2023.3.post1)
Requirement already satisfied: frozendict>=2.3.4 in
/usr/local/lib/python3.11/site-packages (from yfinance) (2.4.6)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.11/site-
packages (from yfinance) (3.17.8)
Requirement already satisfied: beautifulsoup4>=4.11.1 in
/usr/local/lib/python3.11/site-packages (from yfinance) (4.12.2)
Requirement already satisfied: curl_cffi<0.14,>=0.7 in
/usr/local/lib/python3.11/site-packages (from yfinance) (0.13.0)
Requirement already satisfied: protobuf>=3.19.0 in
/usr/local/lib/python3.11/site-packages (from yfinance) (5.29.2)
Requirement already satisfied: websockets>=13.0 in
/usr/local/lib/python3.11/site-packages (from yfinance) (15.0.1)
Requirement already satisfied: cffi>=1.12.0 in /usr/local/lib/python3.11/site-
packages (from curl_cffi<0.14,>=0.7->yfinance) (1.15.1)
Requirement already satisfied: certifi>=2024.2.2 in
/usr/local/lib/python3.11/site-packages (from curl_cffi<0.14,>=0.7->yfinance)
(2025.11.12)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/site-
packages (from beautifulsoup4>=4.11.1->yfinance) (2.4.1)
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/site-
packages (from cffi>=1.12.0->curl_cffi<0.14,>=0.7->yfinance) (2.21)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.11/site-packages (from pandas>=1.3.0->yfinance) (2.8.2)
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-
packages (from pandas>=1.3.0->yfinance) (2023.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-
packages (from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.16.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance) (3.1.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-
packages (from requests>=2.31->yfinance) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/site-packages (from requests>=2.31->yfinance)
(1.26.15)

```

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>. Use the --root-user-action option if you know what you are doing and want to suppress this warning.

```
[2]: import requests
import praw
from datetime import datetime, timedelta
import matplotlib.pyplot as plt
import pandas as pd
from collections import defaultdict
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError
import yfinance as yf
import seaborn as sns
sns.set()
```

Key Considerations

1. Quota Management: Be mindful of the API quota limits. Google provides a limited number of requests per day for free. Plan your data retrieval carefully to stay within these limits or you will be forced to stop your workflow for a day.
2. Keyword Selection: The choice of keywords is critical. Think about the terms that were relevant during the time of the financial event. For our case study on the 2020 crash, we used terms like “recession,” “economic crisis,” “inflation,” etc.
3. Channel Selection: Choose reputable and relevant channels that provide consistent and high-quality content.
4. Data Analysis: After retrieving the data, you’ll analyze the frequency of the keywords and correlate this with financial metrics like the S&P 500 index.

```
[ ]: API_KEY = "YOUR_API_KEY"
CHANNEL_NAMES = ['cnbc'] # You can include your selection of Youtube finance_
↪ channels
CHANNEL_IDS = {} # Dictionary to store channel names and their IDs

# Function to get channel ID by channel name
def get_channel_id_by_name(channel_name, api_key):
    url = f"https://www.googleapis.com/youtube/v3/search?
↪ part=snippet&type=channel&q={channel_name}&key={api_key}"
    response = requests.get(url)
    data = response.json()
    return data['items'][0]['snippet']['channelId']

# Get channel IDs for all channel names
for channel_name in CHANNEL_NAMES:
    channel_id = get_channel_id_by_name(channel_name, API_KEY)
    CHANNEL_IDS[channel_name] = channel_id

# Print channel names and their IDs
for channel_name, channel_id in CHANNEL_IDS.items():
    print(f"{channel_name}: {channel_id}")
```

```
[3]: # Define parameters
KEYWORDS = ['recession', 'unemployment', 'crash', 'stimulus', 'crisis', 'inflation'] # List of keywords to search for. You can include your selection of relevant keywords
START_DATE = '2020-01-01T00:00:00Z'
END_DATE = '2020-05-30T23:59:59Z'

[4]: # Initialize YouTube API
def initialize_youtube(api_key):
    youtube = build('youtube', 'v3', developerKey=api_key)
    return youtube

# Get videos from a channel
def get_videos(youtube, channel_id, start_date, end_date):
    request = youtube.search().list(
        part='snippet',
        channelId=channel_id,
        publishedAfter=start_date,
        publishedBefore=end_date,
        maxResults=50,
        type='video'
    )
    videos = []
    while request:
        response = request.execute()
        videos.extend(response['items'])
        request = youtube.search().list_next(request, response)
    return videos

# Get comments from a video
def get_comments(youtube, video_id):
    comments = []
    request = youtube.commentThreads().list(
        part='snippet',
        videoId=video_id,
        maxResults=100
    )
    while request:
        try:
            response = request.execute()
            comments.extend(response['items'])
            request = youtube.commentThreads().list_next(request, response)
        except HttpError as e:
            if e.resp.status == 403 and 'commentsDisabled' in str(e):
                print(f"Comments are disabled for video ID: {video_id}")
                break
            else:
                pass
```

```

        raise e
    return comments

# Filter data and count keyword mentions
def filter_and_count_comments(comments, keywords, start_date, end_date):
    keyword_counts = defaultdict(int)
    start = datetime.strptime(start_date, '%Y-%m-%dT%H:%M:%SZ')
    end = datetime.strptime(end_date, '%Y-%m-%dT%H:%M:%SZ')
    current_date = start

    while current_date <= end:
        keyword_counts[current_date.strftime('%Y-%m-%d')] = 0
        current_date += timedelta(days=1)

    for comment_thread in comments:
        comment = comment_thread['snippet']['topLevelComment']['snippet']
        comment_date = datetime.strptime(comment['publishedAt'], '%Y-%m-%dT%H:%M:%SZ').strftime('%Y-%m-%d')
        comment_text = comment['textDisplay'].lower()
        if any(keyword in comment_text for keyword in keywords):
            keyword_counts[comment_date] += 1

    return keyword_counts

def plot_keyword_counts(keyword_counts, channel_name):
    dates = list(keyword_counts.keys())
    counts = list(keyword_counts.values())
    df = pd.DataFrame({'Date': dates, 'Count': counts})
    df['Date'] = pd.to_datetime(df['Date'])
    df = df.set_index('Date')
    df.sort_index(inplace=True)

    plt.figure(figsize=(14, 7)) # Adjust the figure size for better readability
    ax = df.plot(kind='bar', legend=False, width = 0.8)
    ax.set_title(f'Keyword Mentions per Day in {channel_name}')
    ax.set_xlabel('Date')
    ax.set_ylabel('Count of Keywords')

    # Set x-ticks at regular intervals and rotate labels for better readability
    ax.set_xticks(range(0, len(df.index), 14))
    ax.set_xticklabels(df.index.strftime('%Y-%m-%d')[::14], rotation=45,
    ha='right')

    #plt.tight_layout()
    plt.show()

```



```
[ ]: # Main script
# THIS CELL CAN TAKE SIGNIFICANT TIME TO COMPLETE EVEN CLOSE TO 20 MINUTES.
# PLEASE BE PATIENT.

# Initialize the YouTube API client
youtube = initialize_youtube(API_KEY)

# Fetch videos from the specified channel within the date range
videos = get_videos(youtube, channel_id, START_DATE, END_DATE)

# Initialize a list to store all comments
all_comments = []

# Fetch comments for each video
for video in videos:
    video_id = video['id']['videoId']
    try:
        comments = get_comments(youtube, video_id)
        all_comments.extend(comments)
    except HttpError as e:
        print(f"An error occurred: {e}")

# Filter and count keywords in comments
keyword_counts = filter_and_count_comments(all_comments, KEYWORDS, START_DATE,
↵END_DATE)
```

In the cell below, you can see what a response looks like. You can use your imagination to think of how the data can be used. For example, the `likeCount` key can be used to assess whether a specific comment should be weighted more than the rest. The `totalReplyCount` values can be used to identify controversial, favorite, or interesting topics.

```
[ ]: comments[0]
```

```
[5]: import json

# Load the local JSON file directly into the variable 'all_comments'
with open('youtube_comments.json', 'r') as f:
    all_comments = json.load(f)

print(f"Loaded {len(all_comments)} comments from file.")

# Filter and count keywords in comments
keyword_counts = filter_and_count_comments(all_comments, KEYWORDS, START_DATE,
↵END_DATE)
```

Loaded 2101 comments from file.

```
[6]: # Preparing dataset for visualization
df_keywords = pd.DataFrame(keyword_counts.items()).set_index(0)
df_keywords.index = pd.to_datetime(df_keywords.index)
df_keywords.sort_index(inplace=True)
df_keywords.columns = ['count']
df_keywords['count'] = df_keywords['count'].astype(int)
df_keywords = df_keywords.resample('D').sum()

rolling_sum = df_keywords.loc["2020-05-30"].rolling('7D').sum()

# Downloading S&P500 as a benchmark
gspc = yf.download('^GSPC', start='2020-01-01', end='2020-05-30')['Close']

# Plotting
fig, ax1 = plt.subplots(figsize=(16, 7))

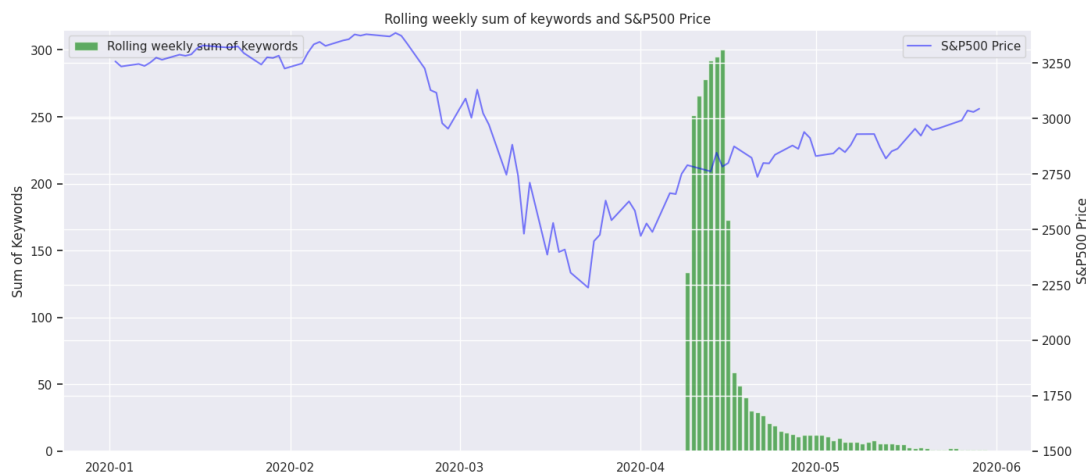
# Plotting the keywords data
#ax1.plot(df_keywords.loc["2020-05-30"].rolling('7D').sum(), color='green',
#         alpha=0.6, label='Rolling weekly sum of keywords')
ax1.bar(rolling_sum.index, rolling_sum.values.flatten(), color='green', alpha=0.
        6, label='Rolling weekly sum of keywords')
#ax1.set_ylim(-1, 40)
ax1.set_ylabel('Sum of Keywords')

# Creating a twin axis to plot S&P500 data
ax2 = ax1.twinx()
ax2.plot(gspc, color='blue', alpha=0.5, label='S&P500 Price')
ax2.set_ylim(1500, 3400)
ax2.set_ylabel('S&P500 Price')

# Adding legends
ax1.legend(loc='upper left')
ax2.legend(loc='upper right')

plt.title('Rolling weekly sum of keywords and S&P500 Price')
plt.show()
```

[*****100%*****] 1 of 1 completed



The graph above is an example of how one can quantify sentiment. The easiest way to do so is by counting the number of words in our choice, and given that some words are not easily found unless there is a reason, we can trust that the excess mentioning of such words indicates the sentiment at that time. The green bars represent the sum of keywords like “recession,” “economic crisis,” and “inflation” found in YouTube comments, indicating heightened public concern. The blue line tracks the S&P 500 index price, highlighting the market’s reaction during the same period. As the keyword mentions continue to rise after March 2020, coinciding with the steep decline in the S&P 500, we observe the rising concern of the people and the impact of the crash on overall sentiment.

Exercise 1

Choose an asset and a period when significant events for this asset occurred. Identify the keywords you want to count and the channel(s) that are appropriate for your analysis. Use the code provided and follow the same workflow as above. Plot the results (you may need to do serious modifications in the code) and paste it in the forum along with a “what we see” analysis.

1.2.2 2.2 Reddit

Reddit is a popular social news and discussion website where users can share content and engage in conversations on various topics, including cryptocurrency. For a financial engineering student, Reddit data can give valuable examples as to how to gain insight into market sentiment and trends.

In this section, we’ll explore how to use the Reddit API to collect and analyze data related to cryptocurrency discussions. The Reddit API allows developers to access Reddit’s database of posts, comments, and other content programmatically. To use the API, you’ll need to create a Reddit account and register an application to obtain the necessary credentials (client ID, client secret, and user agent). At this point, you are advised to use AI instructions in order to complete this step.

Our example focuses on analyzing sentiment in cryptocurrency-related subreddits over the past month. This is due to the limitations of Reddit’s API and the fact that it was not created for data analysis but merely as a tool for subreddit moderators. The code considers only the last month (the last month starting from the time you run the code cells), and for this reason, the results are

expected to be different every time you run the code.

You are **STRONGLY** advised to not spam the cell that downloads the comments as your app will be restricted from reaching these subreddits, even though that can be corrected by creating a new app. This is a good reminder to students that sentiment analysis data do not come for free. We are bounded by API restrictions and the absence of free historical data.

It's also important to note that while we often expect sentiment analysis to align closely with price movements, this correlation isn't always perfect. Factors such as market manipulation, external news, or regulatory changes can cause divergences between sentiment and price. However, sentiment analysis remains a valuable tool for understanding market psychology and potential price directions.

```
[ ]: # Initialize the Reddit instance

reddit = praw.Reddit(
    client_id='CLIENT_ID',
    client_secret="CLIENT_SECRET",
    user_agent='USER_AGENT',
    check_for_async=False
)

# Example of fetching the top posts from a subreddit
submissions = reddit.subreddit('finance').top(limit=5)
for submission in submissions:
    print(f"Title: {submission.title}")
    print(f"Score: {submission.score}")
    print(f"URL: {submission.url}")
    print("="*40)
```

```
[ ]: # Define subreddits
subreddits = ['cryptocurrency']

# Define bull and bear sentiment keywords
bull_keywords = ['bull', 'bullish', 'moon', 'lambo', 'hodl', 'buy', 'long',
    ↳ 'uptrend', 'breakout', 'rally']
bear_keywords = ['bear', 'bearish', 'crash', 'dip', 'sell', 'short',
    ↳ 'downtrend', 'correction', 'dump', 'fud']

# Function to get comments from a subreddit
def get_comments(subreddit_name, start_date):
    subreddit = reddit.subreddit(subreddit_name)
    comments = []
    for submission in subreddit.new(limit=None):
        if submission.created_utc < start_date:
            break
        submission.comments.replace_more(limit=0)
        for comment in submission.comments.list():
            comments.append({
```

```

        'text': comment.body,
        'created_utc': datetime.fromtimestamp(comment.created_utc),
        'subreddit': subreddit_name
    })
    return comments

# Get comments from the last month
end_date = datetime.now()
start_date = end_date - timedelta(days=30)
all_comments = []

for subreddit in subreddits:
    all_comments.extend(get_comments(subreddit, start_date.timestamp()))

```

NLTK (Natural Language Toolkit) is a leading platform for building Python programs to work with human language data. In our analysis, we use several NLTK components:

- **VADER** (Valence Aware Dictionary and sEntiment Reasoner) lexicon: A rule-based sentiment analysis tool specifically attuned to sentiments expressed in social media.
- **SentimentIntensityAnalyzer**: A VADER-based tool that provides sentiment scores for text.
- **word_tokenize**: A function that splits text into individual words or tokens.

These tools allow us to process and analyze the sentiment of Reddit comments effectively, providing insights into the overall mood of cryptocurrency discussions.

```

[ ]: import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
from nltk.tokenize import word_tokenize

# Download the VADER lexicon
nltk.download('vader_lexicon')
nltk.download('punkt')
nltk.download('punkt_tab')

# Initialize the sentiment analyzer
sia = SentimentIntensityAnalyzer()

```

Exercise 2

Explain the differences between the `.split(" ")` function and `word_tokenize` by using the documentation. Display those differences by using a paragraph. Explain with simple words what tokenization is in the NLP context.

```

[ ]: # Preparing the dataset for visualization
text_df = pd.DataFrame(all_comments) # Create a DataFrame from the comments

text_df['tokenized'] = text_df['text'].apply(word_tokenize) # Tokenize the
↪ comments

```

```

text_df['tokenized'] = text_df['tokenized'].apply(lambda x: [word.lower() for
    ↪word in x]) # Convert words to lowercase
3669
text_df['bull_count'] = text_df['tokenized'].apply(lambda x: sum(1 for word in
    ↪x if word in bull_keywords)) # Count bull keywords
text_df['bear_count'] = text_df['tokenized'].apply(lambda x: sum(1 for word in
    ↪x if word in bear_keywords)) # Count bear keywords

text_df['sia'] = text_df['text'].apply(lambda x: sia.
    ↪polarity_scores(x)['compound']) # Calculate sentiment using NLTK

text_df.set_index('created_utc', inplace=True) # Set the index to the
    ↪created_utc column
text_df.sort_index(inplace=True) # Sort the DataFrame by the index

```

```

[7]: import pandas as pd

# Load the CSV
text_df = pd.read_csv('reddit_comments.csv')

# Convert the date column to datetime objects
text_df['created_utc'] = pd.to_datetime(text_df['created_utc'])
start_date = text_df["created_utc"].min().date()
end_date    = text_df["created_utc"].max().date()

print(f"Dates initialized from CSV:\nStart: {START_DATE}\nEnd:    {END_DATE}")

# Set the index to 'created_utc' and sort (required for the rolling mean plots
    ↪later)
text_df.set_index('created_utc', inplace=True)
text_df.sort_index(inplace=True)

# OPTIONAL: The columns 'tokenized', 'bull_count', 'bear_count', and 'sia'
# are already in the CSV. You can skip the lines in your notebook that
    ↪calculate them.
print(text_df.head())

```

```

Dates initialized from CSV:
Start: 2020-01-01T00:00:00Z
End:    2020-05-30T23:59:59Z

```

```

text
created_utc
2024-07-01 09:17:47  \n**TL;DR**\n\nMetaplanet's recent Bitcoin pu...  \
2024-07-01 09:20:20  tldr; Metaplanet, a Tokyo Stock Exchange-liste...
2024-07-01 09:44:39                                     Me likey
2024-07-01 10:20:49      Governments shouldn't be allowed to buy crypto
2024-07-01 10:28:17                                     Why? Isnt it a free market

```

```

                                subreddit
created_utc
2024-07-01 09:17:47  cryptocurrency \
2024-07-01 09:20:20  cryptocurrency
2024-07-01 09:44:39  cryptocurrency
2024-07-01 10:20:49  cryptocurrency
2024-07-01 10:28:17  cryptocurrency

```

```

                                tokenized
created_utc
2024-07-01 09:17:47  ['*', '*', 'tl', ';', 'dr', '*', '*', '>', 'me... \
2024-07-01 09:20:20  ['tldr', ';', 'metaplanet', ',', 'a', 'tokyo',...
2024-07-01 09:44:39  ['me', 'likey']
2024-07-01 10:20:49  ['governments', 'should', 'n't', 'be', 'allowe...
2024-07-01 10:28:17  ['why', '?', 'isnt', 'it', 'a', 'free', 'market']

```

```

                                bull_count  bear_count      sia
created_utc
2024-07-01 09:17:47              0          0  0.4939
2024-07-01 09:20:20              0          0  0.9287
2024-07-01 09:44:39              0          0  0.0000
2024-07-01 10:20:49              1          0  0.0000
2024-07-01 10:28:17              0          0 -0.4023

```

```

[8]: # Downloading BTCUSD price as a cryptomarket proxy.
      btcusd = yf.download('BTC-USD', start=start_date, end=end_date)['Close'] #_
      ↪Download Bitcoin price data

```

```

[*****100%*****] 1 of 1 completed

```

```

[9]: # Superimposing the price on the smoothened sentiment score

fig, ax1 = plt.subplots(figsize=(14, 5)) # Create a figure and axis
ax2 = ax1.twinx() # Create a second axis that shares the same x-axis

smoothened_sia = text_df[['sia']].resample('D').mean().rolling('7D').mean() #_
↪Calculate the rolling mean of SIA
ax1.plot(smoothened_sia, color='green', label='Sentiment Analysis Score') #_
↪Plot the SIA mean
ax2.plot(btcusd, color='blue', label='BTC-USD') # Plot the Bitcoin price

ax1.set_ylabel('SIA Sentiment') # Set the label for the first axis
ax2.set_ylabel('BTC-USD Price') # Set the label for the second axis
ax1.set_xlabel('Date') # Set the label for the x-axis

# Get the handles and labels from both axes
lines_1, labels_1 = ax1.get_legend_handles_labels()

```

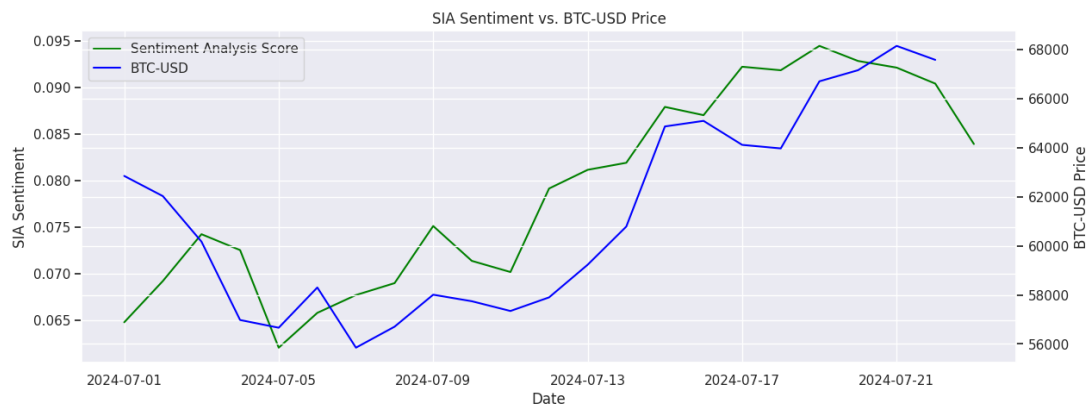
```

lines_2, labels_2 = ax2.get_legend_handles_labels()

# Combine the handles and labels, and create a legend
ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper left')

plt.title('SIA Sentiment vs. BTC-USD Price') # Set the title of the plot
plt.show() # Show the plot

```



Let's do the same as above, but now, let's choose our own keywords in estimating the sentiment. We will aggregate the count of each bullish word and each bearish word. In order to have a sentiment estimation, we will use the difference bull - bear. This difference will be smoothed by a 7-day window (as in the SIA case) before we superimpose the BTCUSD price.

```

[10]: keywords_bull_bear = text_df[['bull_count', 'bear_count']].resample('D').sum()
bull_minus_bear = keywords_bull_bear['bull_count'] -
↳ keywords_bull_bear['bear_count']
bull_minus_bear = bull_minus_bear.rolling('7D').mean()

```

```

[11]: fig, ax1 = plt.subplots(figsize=(14, 5)) # Create a figure and axis
ax2 = ax1.twinx() # Create a second axis that shares the same x-axis

ax1.plot(bull_minus_bear, color='green', label='Bull - Bear') # Plot the
↳ difference
ax2.plot(btcusd, color='blue', label='BTC-USD') # Plot the Bitcoin price

ax1.set_ylabel('Bull - Bear counts') # Set the label for the first axis
ax2.set_ylabel('BTC-USD Price') # Set the label for the second axis
ax1.set_xlabel('Date') # Set the label for the x-axis

# Get the handles and labels from both axes
lines_1, labels_1 = ax1.get_legend_handles_labels()

```



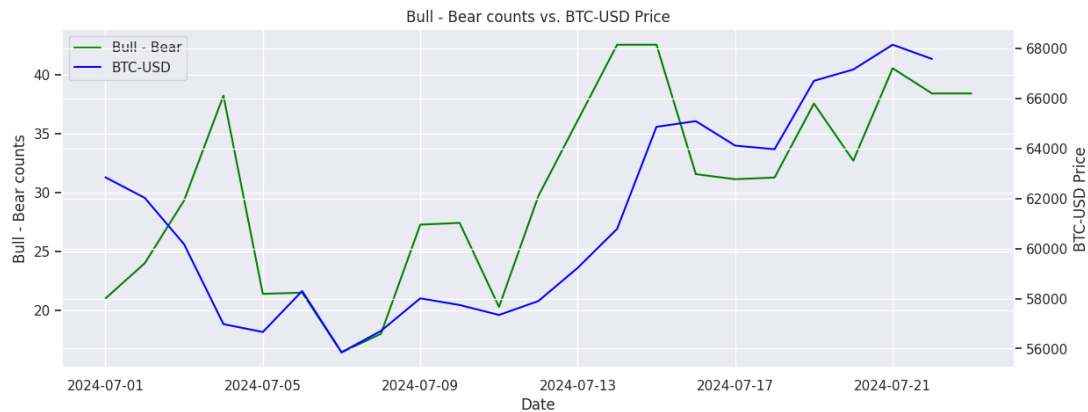
```

lines_2, labels_2 = ax2.get_legend_handles_labels()

# Combine the handles and labels, and create a legend
ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper left')

plt.title('Bull - Bear counts vs. BTC-USD Price') # Set the title of the plot
plt.show() # Show the plot

```



Exercise 3

In the Reddit section, we have downloaded all the comments during a specific period for one subreddit. By using the code provided, you can select the keywords (given the subreddit we have already selected) that will help you search additional trends (specific altcoins, specific meme coins, etc). Use both the `nltk` and the manual way of estimating sentiment. Paste your results in the forums.

1.3 3. Conclusion

In conclusion, this lesson has demonstrated the powerful role that social media data can play in financial analysis. We've explored how to retrieve and analyze data from major platforms like YouTube and Reddit, using API interactions and employing simple sentiment analysis techniques. These methods provide valuable insights into market sentiment, public opinion, and the trends that can impact financial markets.

The examples given are significant in the sense of what one can achieve with a **free** API. Keep in mind that data for sentiment analysis are hard to find and most likely expensive. But by carefully choosing your period and channels, subreddits, tweets, etc., you will be able to extract, quantify, and visualize the sentiment.

References

- Boyd, Danah M., and Nicole B. Ellison. "Social Network Sites: Definition, History, and Scholarship." *Journal of Computer-Mediated Communication*, vol. 13, no. 1, 2007, pp. 210-230.

- “Social media.” *Cambridge Dictionary*, Cambridge University Press. <https://dictionary.cambridge.org>. Accessed 18 Nov. 2024.
- “Social media.” *Merriam-Webster.com Dictionary*, Merriam-Webster, <https://www.merriam-webster.com>. Accessed 18 Nov. 2024.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.